

From Juggling to Airports

Part B - Final Report

Carla Francisca Henriquez-Venegas

04/08/2026

Project Supervisor: Anthony Mays

Declaration

In submitting this work I am indicating that I have read the University's Academic Integrity Policy. I declare that all material in this assessment is my own work except where there is clear acknowledgement and reference to the work of others. I give permission for this work to be reproduced and submitted to other academic staff for educational purposes.

Abstract

Aircraft turnaround services compete for a resource that is fixed in the short-term: a fixed number of available bays to host the service simultaneously. This project models bay occupancy as a discrete-time Markov chain, adapting the state representation from the mathematics of juggling. Two models were built, the first with a single aircraft type and equally likely events, that was used to validate the methods. The model was extended to two aircraft types with different turnaround service durations and arrival probabilities estimated from 51 days of Adelaide Airport flight data.

The extended Markov chain has 39 reachable states and a unique stationary distribution, which converged to by the simulations from opposite starting states that were performed. From the stationary distribution, the system stays empty 34.03% of the time, runs at 31.84% utilisation, and rejects 1.80% of arrivals. Both aircraft types the same theoretical rejection probability since both occupied one bay. However, a 737 returns 13.19% more net revenue per beat per bay than an e190. The sensitivity analysis showed that rejection rates increased non-linearly as arrival rates did.

1 Introduction

Every flight requires a service turnaround. During this process the aircraft is unloaded, cleaned, refuelled, catered and reloaded before its next departure [1, p.2]. The number of bays available for simultaneous service is limited in the short-term, and each occupied bay remains unavailable for the duration of its service.

Under this model framework, an arrival cannot be admitted when bays are fully occupied. Then, every rejected aircraft represents a hypothetical revenue that would be generated if that arriving aircraft were serviced. Questions may arise as *how often the system is full*, and *how many aircraft are being serviced and rejected* and *how much revenue is associated with them*, these questions will be answered with the the long-run behaviour of the system.

Existing research approaches the problem from an individual turnaround service perspective, [12, p.32] surveys existing models, where none of which approaches from the perspective of modelling bays across the airport system as a whole.

This investigation models that system through the mathematics of juggling, aiming to answer whether a discrete-time Markov chain built from real arrival rates can quantify the long-run behaviour of an airport turnaround system. Specifically, it aims to answer the following questions:

1. How is bay occupancy distributed in the long run?
2. If simulations are performed, do they agree with the theoretical stationary distribution?
3. How many aircraft arrivals are expected to be serviced or rejected?
4. How do changes in the arrival rate affect the long-run rejection rate of the system?
5. What is the estimated revenue of the system and per aircraft?

The model was built in two stages: first a small binary model with equally likely events, used to verify the methodological steps. Next, an extended model, adding two aircraft types with different service turnaround times, arrival probabilities estimated from real data, an economic layer which transforms serviced and rejected aircraft into net revenue, and a sensitivity analysis over the arrival probabilities.

Overall, both models produce a unique stationary distribution, its theoretical results were compared with two simulations performed per model, each one of 1,000,000 steps, starting from empty and full states, these values were used to estimate long-run bay occupancy, aircraft service and rejection rates. In addition two methods were added to the extension model: economic contribution and the effect of changing arrival rates.

2 Background

2.1 Juggling Theory

The mathematical study of juggling began between 1980 and 1985, when the first *juggling notations* were introduced [3, p.507], [7, p.18]. One of the main concepts arising is the **siteswap**. A siteswap indicates the sequential order of catches and throws for objects being juggled in successive beats [7, p.106], or from a diagrammatic perspective, it can be interpreted as the sequence of edges from a graph loop [4, p.429].

Another relevant concept is the **state**, which stores the sequence of objects expected to land at any given beat, it can be represented as:

$$x_1, x_2, \dots, x_h, \quad \text{with } h = \text{maximum horizon of time}$$

The **shifting mechanism**, as [4, p.429-430] suggests, occurs when a transition between states take place, it implies by shifting the state by one position to the left, removing its first entry and appending a 0 as its last position:

$$x_1, x_2, x_3, \dots, x_{h-1}, x_h \longrightarrow x_2, x_3, \dots, x_{h-1}, 0 \quad (2.1)$$

If there is an arrival during the current beat, it should be placed in an available position [4, p.429-430], as long as there is room for it.

Therefore, a **juggling pattern** is a repetitive sequence of **discrete, non-negative values**, representing **throws** whose **height** symbolises the *time steps* elapsed between an object being thrown and caught again [5, p.8] [7, p.107].

To analyse **simple patterns of juggling**, which are the focus of this research, it is required to assume an idealised framework, where objects are thrown constantly per beat at evenly spaced rhythm, the objects have been juggled continuously and uninterrupted, being immediately thrown again upon its capture [5, p.105], [6], [3].

Under these assumptions, the systems' evolution depends only on the **current juggling state** [6], which records the objects expected to land within the next h beats.

2.2 Markov chain Theory

A discrete-time Markov chain is a sequence of random variables defined over a **finite state space** that changes in discrete time steps [10, p.213-214], where the state space contains all possible **state configurations** that the system can occupy [15, p.1-2]. A current state may move to the next one in one step of time [6, p.109], according to a **transition probability** [11, p.194-195]. Since the next occupied state cannot be predicted with complete certainty, the system is described as stochastic.

To identify all possible state configurations of a system, an appropriate counting method is required, its selection depends on the state vector values¹:

1. Binary state vector:

- (a) With length h and **fixed** b objects in the system, the state is determined by choosing which b of the h positions are occupied, since they cannot be occupied twice, this definition corresponds to a **combination without replacement** [11, p.29-30]:

$$|J(b, h)| = \binom{h}{b} = \frac{h!}{b!(h-b)!} \quad (2.2)$$

- (b) If b **varies** from 0 to h , the total number of states is obtained by summing equation (2.2) over all possible values of b :

$$|J(0 : h, h)| = \sum_{b=0}^h \binom{h}{b} = 2^h \quad (2.3)$$

2. Non-binary state vector:

- (a) With length h and **fixed** b objects, the same position may be occupied more than once. The entries of the vector satisfy:

$$x_1 + x_2 + \dots + x_h = b$$

This corresponds to a **combination with replacement**, where the number of possible states is given by:

$$|J(b, h)| = \binom{h+b-1}{b} \quad (2.4)$$

- (b) If b **vary** from 0 to the system maximum *capacity*, the entries satisfy:

$$x_1 + x_2 + \dots + x_h \leq \text{capacity}$$

Hence, the count of states is obtained by summing the Equation (2.4) over all values of b :

$$|J(0 : \text{capacity}, h)| = \sum_{b=0}^{\text{capacity}} \binom{h+b-1}{b} = \binom{h+\text{capacity}}{\text{capacity}} \quad (2.5)$$

Under the idealised juggling conditions described previously, the **probability of transitioning** to a next state depends only on the **current state** and not the history of states visited previously. This is known as the **Markov property** [6], and can be written as:

$$P(X_{n+1} = j \mid X_n = i, X_{n-1}, \dots, X_0) = P(X_{n+1} = j \mid X_n = i) \quad (2.6)$$

with n = current beat n , $n+1$ = next beat $n+1$

¹These expressions mentioned before, count **all possible state configurations** satisfying *capacity* constraints. Some states may be unreachable under specific model conditions.

According to [11, p.195-196], [10, p.213-215] each transition probability p between states i and j ($p_{i,j}$) is given by:

$$P(X_{n+1} = j \mid X_n = i) = p_{i,j} \quad (2.7)$$

$$\text{satisfying, } 0 \leq p_{i,j}(n) \leq 1 \text{ for all } i, j, \quad \sum_j p_{i,j}(n) = 1 \text{ for every state } i \quad (2.8)$$

Hence, a juggling system satisfying these idealised assumptions can be represented using a discrete-time Markov chain as a framework.

Note that each $p_{i,j}$ described in 2.7, must satisfy the properties mentioned in 2.8.

Therefore, when collecting each $p_{i,j}$ entries, in rows i and columns j , the resulting squared matrix is called **Markovian** or **stochastic** matrix [11, p.195].

Taking the following structure:

$$P = \begin{bmatrix} p_{i_1 j_1} & p_{i_1 j_2} & \cdots & p_{i_1 j_n} \\ p_{i_2 j_1} & p_{i_2 j_2} & \cdots & p_{i_2 j_n} \\ \vdots & \vdots & & \vdots \\ p_{i_n j_1} & p_{i_n j_2} & \cdots & p_{i_n j_n} \end{bmatrix}. \quad (2.9)$$

The transition probability between states $p_{i,j}$ represents the likelihood of moving from one *state* i to another *state* j , including self-transitions, and takes values ranging from 0 to 1, where $p_{i,j} = 0$ denotes an impossible transition and $p_{i,j} = 1$ denotes complete certainty.

Following the ideas introduced by [6, p.110], from the Markov chain matrix it is possible to compute the long-run proportion of time it spends in each state by solving the non-zero **left eigenvector** v of the transition matrix P , when λ , equals to 1 [11, p.235]:

$$vP = \lambda v, \quad \text{with } \lambda = 1$$

Thus,

$$vP = v \quad (2.10)$$

An eigenvector is a non-zero vector whose direction remains unchanged when multiplied by a matrix. The corresponding eigenvalue λ indicates the factor by which its magnitude is scaled. Therefore, when $\lambda = 1$, multiplication by P leaves the eigenvector v unchanged.

Equivalently, the left eigenvector $\mathbf{v}$ of P can be computed as the right eigenvector of the transposed matrix P^T [11, p.738]:

$$P^T v^T = v^T \quad (2.11)$$

Then the eigenvector is normalised [6, p.110] so their entries comply with:

$$0 \leq v_i \text{ and } \sum_i v_i = 1, \quad \text{with } v_i = \text{theoretical stationary distribution}$$

The result is a unique solution, known as the **stationary distribution** or **steady state** [9, p.46], where each entry represents the long-run **proportion of time** the system spends in each state, regardless of its initial starting distribution [11, p.238].

When simulating, the convergence of a Markov chain describes how the system stabilises [9, p.55], approaching to its theoretical stationary distribution, as simulation steps increase. In this research, empirical convergence is measured using the Residual Sum of Squares, hereafter RSS:

$$\text{RSS}^{(t)} = \sum (OD_t(i) - v_i)^2 \quad (2.12)$$

Where $OD_t(i)$ denotes the **observed distribution** as the proportion of visits to state i up to step t :

$$OD_t(i) = \frac{\text{visit count of state } i \text{ up to step } t}{t} \quad (2.13)$$

Where, v_i denotes the **theoretical stationary distribution** probability assigned to state i .

RSS quantifies the distance between each simulated distribution to the Stationary distribution at each time step t , across all states.

With the theoretical framework established, the next subsection introduces the airport turnaround context in which a Markov chain model will be applied.

2.3 Airport background

Aircraft turnaround operations are the activities required to prepare an aircraft for its next departure [17, p.49], including passenger handling, refuelling, cleaning, catering, and baggage tasks [1, p.2]. These activities form the essential link between airport infrastructure and aircraft activity is crucial when shaping overall operational efficiency [12, p.29].

Delays in turnaround processes are transmitted through airline networks because the same aircraft is scheduled to operate consecutive flights across the day; a late departure at one station becomes a late arrival at the following, which can decrease considerably the available time for the next turnaround service [17, p.48], [12, p.30].

Several approaches to aircraft turnaround operations ignore operational variability, making them too rigid to adapt to changing conditions [1, p.3]. Stochastic approaches address uncertainty more directly: [17, p.52] proposed a stochastic model that represents departure delay to arrival times using probability density functions, while [18, p.3] applied a simulation that models the physical interactions of individual passengers and cleaning crew members inside the aircraft cabin. [12, p.32] surveys these approaches comprehensively, observing that turnaround time is stochastic in nature and fluctuates from flight to flight due to differences in passenger load, fuel requirements, and cargo volume. However, existing approaches mainly focus on modelling the sequence of activities within individual aircraft turnarounds rather than the airport bay occupation system as a whole. In particular, no existing framework models the global bay occupation state space as a bounded Markov chain and investigates whether the system converges towards a stable long-run stationary distribution across all possible bay occupation configurations, a gap that motivates the present research.

3 Model and Methods

3.1 Model overview and scope

This research develops a discrete-time Markov chain model of turnaround services at Adelaide Airport inspired by the mathematics of juggling.

The project was developed in two stages: **Part A-Initial approach** and **Part B-Extended model**.

3.2 Notations and definitions

To clarify the concepts used throughout the project, a list of definitions is provided:

Definition 3.1.

- *beat*: Discrete unit of **time**.
- *capacity*: The maximum number of aircraft that can be serviced **simultaneously**. In both models *capacity* = 3.
- *b*: The **number of planes** currently in the system. Since aircraft may arrive and depart, then $b \in \{0, 1, \dots, \text{capacity}\}$.
- *service time*: The number of *beats* required to complete a turnaround service. Part B values are in Table 7.
- *state*: A finite vector where each entry reflects service completion at any *beat*. Accepted aircraft are positioned in the vector at the slot corresponding to their service time. See Table 7.
- *h*: Number of positions in the *state* vector. Represents the **maximum horizon of time in beats**.
- $|state|$: operation that reflects the *occupancy levels* of the system, computed as:

$$|state| = \sum_{i=1}^h x_i, \quad x_i = \text{state entry } i \quad (3.1)$$

- *occupancy levels*: There are two classifications:
 - **Empty state**: when $|state| = 0$.
 - **Full state**: when $|state| = capacity = 3$.
- *event*: At every beat, one mutually exclusive event occurs with a probability:

Part A - event	Probability (p_{event})
Arrival	p_a
No arrival	$p_{na} = 1 - p_a$

Table 1: Part A - Events and probability notation.

Part B - Event	Probability (p_{event})
Small arrival (e190)	p_s
Large arrival (737)	p_l
No arrival	$p_n = 1 - p_s - p_l$

Table 2: Part B - Events and probability notation.

Estimated values can be seen in Table 12.

- *shifted state*: intermediate *state* that has been shifted as defined in 2.1.

Example. State 10200; capacity=3; h=5.

Occupancy levels are computed using Equation (3.1):

$$\begin{aligned}
|state| &= \sum_{i=1}^5 x_i \\
&= x_1 + x_2 + x_3 + x_4 + x_5 \\
&= 1 + 0 + 2 + 0 + 0 \\
&= 3 \rightarrow \text{full state}
\end{aligned}$$

This state can be interpreted as *one aircraft scheduled to complete its service in the next beat, and two aircraft to complete theirs in three beats from the current beat*.

To summarise the parameters used in both models, Table 3 is presented:

Characteristic	Part A	Part B	Changed?
<i>beat</i> duration(minutes)	10	6	Yes
h	3	5	Yes
Aircraft types	1	2	Yes
Number of <i>events</i>	2	3	Yes
Possible <i>state</i> entries	$\{0, 1\}$	$\{0, 1, 2\}$	Yes
Event probabilities	Equal 0.5	Unequal (estimated from airport data)	Yes
Max. aircraft completion in one <i>beat</i>	1	2	Yes
Max. arrival <i>events</i> per <i>beat</i>	1	1	No
System <i>capacity</i>	3	3	No
Vector Shifting mechanism	one position per <i>beat</i>	one position per <i>beat</i>	No

Table 3: Comparison between the models in Parts A and B.

3.2.1 Relationship between juggling and airport systems

The model adapts elements from the mathematics of juggling to represent an airport turnaround system, an analogy between the two is shown in Table 4:

In both systems, each position of the *state* vector represents an *event* scheduled to occur during a future *beat*. In the juggling setting, an object in the current beat is an object that has landed, whereas under the airport framework, it is the completion of an aircraft turnaround service, which releases one unit of system *capacity*.

A juggling throw is represented by inserting an object into the position corresponding with its throw height. Similarly, an accepted aircraft arrival is inserted into the position corresponding to its *service time*.

Juggling framework	Airport setting	Variable name
Finite state vector	Service-completion schedule	<i>state</i>
Number objects juggled	Number of aircraft in service	<i>b</i>
Throw height	Turnaround service duration	<i>service duration</i>
Maximum throw height	Maximum horizon of time	<i>h</i>
Max. objects to be juggled	Max. number of aircraft in simultaneous service	<i>capacity</i>

Table 4: Analogy between juggling and airport ground operations.

The shift-left mechanism is also preserved; Its implementation is described in 3.5.2, step 2a.

3.3 Part A – Initial approach

The initial model used a binary *state* and assumes two equally likely *events* per *beat*: *arrival* or *no arrival*. Thus:

$$p_{event} = \frac{1}{2} = 0.5$$

The system has $h = capacity = 3$ and number of aircraft b between $0 \leq b \leq h$, with only **one** aircraft type, with a *service duration* = 3 beats.

Although the system could reach full capacity, it could not reject an arrival. Since $h = service\ duration$, shifting any *state* always removed the first entry and appended an empty third position. As at most one aircraft could arrive per beat, the new available position could always place a new arrival.

These assumptions limited the model’s ability to represent more complex scenarios this motivates the extension developed in Part B.

3.3.1 State space construction

Algorithm 3.2. For each $0 \leq b \leq h$:

1. Apply function `nchoosek`, which returns the positions that are different from 0 within the vector state (See A.1.1, lines 2,16).
2. Apply function `zero` to create a vector of length h , filled with 0. Create a binary vector where only the resultant positions from step 1 above, are equal to 1 (See A.1.1, lines 4-20).
3. Merge binary vectors in a matrix (See A.1.2, line 1)

3.3.2 Transition matrix construction

Algorithm 3.3. (Appendix A.2):

1. Apply function `zero` to create a matrix filled with 0 (See A.2.1, line 1).
2. Manually populate the matrix with one probability per valid transition between *states* (See A.2.1, line 3-4).
3. Verify each row from our matrix sums to one (See A.2.2, line 2).

3.3.3 Stationary distribution

The stationary distribution was computed by finding the eigenvector of P^T associated with the eigenvalue closer to one. The eigenvector was normalised so that its entries sum to one and double check using the residual $vP - v$.

A detailed procedure is described for Part B in Algorithm 3.7, the codes are the same.

3.3.4 Simulation

To validate the theoretical stationary distribution v , two simulations of 1,000,000 steps were performed, starting from the empty state 000 and the full state 111. Both simulations used 1992 as the random number seed for reproducibility purposes.

Algorithm 3.4. (Code in Appendix A.4)

1. At each beat, the current state was shifted (Lines 1-4).
2. A random number was generated and an event selected according to the events: arrival or no-arrival (Lines 5-7).
3. For an arrival, a candidate state was generated and either accepted or rejected according to the *capacity* (Lines 9-16).
4. The following state, state visits, accepted and rejected arrivals were recorded. The observed distribution was updated and compared with v using RSS at every beat (Lines 13,16,19-23).

3.4 Part B - Extended model

The extended model differentiates between aircraft types by their frequency of operations at Adelaide Airport. Where domestic and regional destinations go up to 86.54%² of total airport traffic.

Although several types of commercial aircraft types operate at Adelaide Airport, the Boeing 737-800, hereafter as 737 or *large aircraft*, and the Embraer E190, hereafter as e190 or *small aircraft*, were selected because together they accounted for 70.49% (See D.2.3, line 14) of the domestic flights correctly categorised in the dataset. To be precise: 53.50% and 16.99%, respectively (See D.2.3, line 12-13).

Figure 1 illustrates the length differences between the aircraft, showing that the 737 is three metres longer than the e190.

Figure 1: Length differences between aircraft types.

While Table 5 specifies that 737 can carry 78 additional passengers than the e190, which represents an 86.67% increased capacity.

Aircraft type	Passenger capacity
Large (737)	168
Small (e190)	90

Table 5: Aircraft passenger capacities, obtained from Qantas-website seat-maps: 737 [13] and e190 [14].

3.4.1 Model assumptions

- Parameter assumptions are in Table 6:

Parameter	Value	Meaning
<i>beat</i>	6 minutes	1 beat → 6 minutes
<i>h</i>	5	Vector length
<i>capacity</i>	3	$ state \leq 3$
<i>b</i>	Up to 3	$b \in \{0, 1, 2, 3\}$

Table 6: Parameter values used in Part B.

- An available bay is assigned to an aircraft upon arrival, no aircraft can depart without being serviced.
- Every plane, and event, has a fixed *service time*, which can be translated into an initial *state* representation, as per Table 7.

Event type	Time in minutes	Time in beats	Initial representation state
Small arrival (e190)	18	3	00100
Large arrival (737)	30	5	00001
No arrival	-	0	00000

Table 7: Turnaround service times and arrival representations used in Part B.

²Calculated from [16], passenger-destination values from March 2025 to February 2026, inclusive

- Planes carry **only** passengers and their allowed baggage (No freight), at 100% of their passengers **capacity**.
- Passenger capacity as per Table 5.
- Regardless of the type of plane, the number of staff is **fixed**.
- Time homogeneous: p_{event} and transition rules remain constant throughout the simulation. Although the transition probabilities may differ between states, they remain unchanged from one beat to another.

3.4.2 Model constraints

- Only **two types** of aircraft are considered: large and small.
- Events in different beats are independent and **mutually exclusive**, then at most one arrival occur per beat.
- Event probabilities are **constant** over time, and have different values between them (See Table 12)
- A state is admissible if $|state| \leq 3$, b cannot exceed the system's *capacity*
- An event may be accepted only after the state has shifted.

As a consequence of a single arrival per *beat* and fixed *service times*, at most two aircraft can be scheduled to finish during the same *beat*, hence **no reachable state** contains an entry larger than 2.

3.5 Model implementation and construction

3.5.1 Parameter estimations - data

Arrival data was gathered from the Adelaide Website³ once per day for the **previous day of operations**, so that every flight has reached its **final status** (*landed, cancelled, other*).

Rates were estimated from 51 days, between 11/06/2026 to 02/08/2026, comprising 4,897 records, cleaned, categorised and examined using Python. In that period, 2 days were missed, but since estimated p_{event} , as per Equation 3.4, divides total arrivals by the number of observed days, 2 missing days do not bias the rate.

With Day = 24 hours; Hour = 60 min; Beat = 6 min. Then,

$$\text{total beats} = \frac{\text{hours day} \times \text{minutes per hour}}{\text{beat}} \quad (3.2)$$

$$\text{total observed days} = 51 \quad (3.3)$$

From Equation (3.2) and (3.3), arrival rates were estimated as follows (See code D.2.4, lines 5-7):

$$p_{event} = \begin{cases} \frac{\text{Arrival count}_{aircraft}}{\text{total observed days}}, & p_s \text{ or } p_l \\ \frac{\text{total beats}}{1 - p_s - p_l}, & p_n \end{cases} \quad (3.4)$$

3.5.2 State Space: construction and reachability

Algorithm 3.5.

1. States generation: Create all states of length $h = 5$ that meet *capacity* criteria (See B.1.1, lines 2-15).
2. Reachability search: Starting from the empty state 00000
 - (a) Generate the *shifted state* as described in Equation (2.1) (See B.1.2, line 5).

³<https://adelaideairport.com.au/flight-information/>

Example.

$$00001 \rightarrow 0001_ \rightarrow 00010$$

(b) Generate the *candidate state*, expanding the *shifted state* into the 3 events:

$$\text{Shifted state} = \begin{cases} \text{Shifted state} + \mathbf{00100}, & \text{if small arrives} \\ \text{Shifted state} + \mathbf{00001}, & \text{if large arrives} \\ \text{Shifted state} + \mathbf{00000}, & \text{if no-arrival} \end{cases} \quad (3.5)$$

Example. From (3.5):

$$00010 = \begin{cases} 00110, & \text{if small arrives} \\ 00011, & \text{if large arrives} \\ 00010, & \text{if no-arrival} \end{cases}$$

(c) Check *candidate state's capacity* (See B.1.2, line 9), according to criteria in Equation (3.1), that can be restated as:

$$\text{candidate state} = \begin{cases} |\text{candidate state}| \leq 3, & \text{Admissible state} \\ |\text{candidate state}| > 3, & \text{Non-admissible state} \end{cases} \quad (3.6)$$

(d) If the *candidate state* is **admissible** is added to *possible states* and *reachable states* (See B.1.2, line 17-22).

If it is **non-admissible**, discard it.

3. Repeat **step 2** for every new *state* until no new states are found.

Example. Starting from State 2 00001, as per Figure 2 on the right. Applying Equation (3.1):

$$\begin{aligned} |\text{State 7}| &= |00110| \\ &= 0 + 0 + 1 + 1 + 0 \\ &= 2 \leq 3 \rightarrow \text{Admissible state} \end{aligned}$$

$$\begin{aligned} |\text{State 8}| &= |00011| \\ &= 0 + 0 + 0 + 1 + 1 \\ &= 2 \leq 3 \rightarrow \text{Admissible state} \end{aligned}$$

$$\begin{aligned} |\text{State 9}| &= |00010| \\ &= 0 + 0 + 0 + 1 + 0 \\ &= 1 \leq 3 \rightarrow \text{Admissible state} \end{aligned}$$

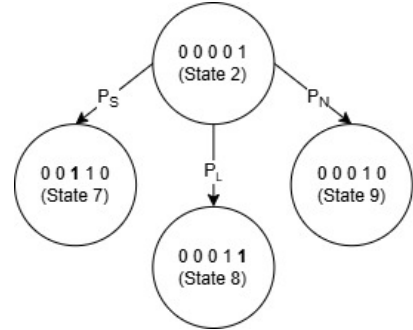


Figure 2: Checking admissibility.

3.5.3 Transition matrix

For each current state i , the 3 events in Table 2 generate states j , the probability of each event is added to the matrix in entry $p_{i,j}$.

If an arrival exceeds the *capacity*, it is rejected and remains outside the system. Therefore, the system produces the same state as if **no arrival event**. However, its probability $p_{i,j}$ is still assigned to the matrix.

Algorithm 3.6.

1. Initialise squared transition matrix P matrix of zeros using **zeros** (See B.2.1, line 1).
2. For a *state* i , in *valid states*, compute its *shifted state* as Equation (2.1) (See B.2.1, lines 2-4).
3. Use **find** to identify the row j containing the *shifted state* in *valid states* (See B.2.1, line 7):
 - (a) If it is not found, it means it is unreachable.

- (b) If it is found, update the entry $p_{i,j}$ adding p_n (See B.2.1, line 12).
- 4. Generate the *candidate state* by applying the remaining events to the *shifted state*, as per Equation 3.5 (See B.2.1, lines 14-15)
- 5. Assess whether $|candidate\ state| \leq 3$ it is satisfied. If accepted, update the entry of P :
 - (a) Small arrival: $p_{i,j} \rightarrow p_{i,j} + p_s$
 - (b) Large arrival: $p_{i,j} \rightarrow p_{i,j} + p_l$.

Note that $p_{i,j}$ may accumulate **more than one event probability**. Since more than one event may produce the same next *state*.

- 6. Repeat for every *state* in *valid states*.

Finally, verify the rows of P sum to one (See B.2.2, line 2).

Example. Following with Figure 2, apply Equation (3.5) to generate the next candidates for State 2 00001:

$$\begin{aligned}
 \text{Shifted state} &= 00001 \rightarrow 00010 \\
 \text{State 2} - 00001 &= \begin{cases} \text{State 7} - 00110, & P_{2,7} = p_s \\ \text{State 8} - 00011, & P_{2,8} = p_l \\ \text{State 9} - 00001, & P_{2,9} = p_n \end{cases} \\
 \text{Verifying} \quad P_{2,7} + P_{2,8} + P_{2,9} &= p_s + p_l + p_n = 1
 \end{aligned}$$

3.5.4 Theoretical stationary distribution: computing and validation

Once P was verified as a valid transition matrix, the long-run behaviour of the system is given by the **stationary distribution**, satisfying Equation (2.11).

For the model, every valid state can reach the empty state through successive *no-arrival* events, while every included state was originally generated from the empty state.

Algorithm 3.7.

1. Apply **eig** to P^T , to obtain the **eigenvalues** and **eigenvectors** of P^T . Since **eig** computes the right eigenvectors, as per Equation (2.11) (See B.3.1, line 1).
2. Extract the **eigenvalues** from the diagonal matrix into a column vector, using **diag** (See B.3.1, line 4).
3. Define a *tolerance* threshold to validate how close the eigenvector is to 1, to be accepted (See B.3.1, line 5,20).
4. Find the column(s) whose eigenvalues accepted, using **find** (See B.3.1, line 5):
 - (a) If more than one are found (See B.3.1, line 7-8), is not possible to compute the TSD.
 - (b) If only one is found, we proceed with it.
5. Extract the column **eigenvector** $\mathbf{v}$, using the chosen eigenvalue, only its real part is retained (See B.3.1, line 11).
6. Normalise the eigenvector so its entries sum to one (See B.3.1, line 12).

The resultant values are the **stationary distribution probabilities**.

7. Verify the results by applying **norm** to transform $|\mathbf{v}^T P - \mathbf{v}^T|$ into a single value (See B.3.1, line 17), which is accepted when it is below the tolerance (See B.3.1, lines 20-21).

3.5.5 Simulation and empirical validation

To assess the theoretical results $\mathbf{v}$ from 3.5.4, two simulations of 1,000,000 steps were performed, starting from:

An agreement between theoretical stationary distribution and simulations is used to verify whether the model reproduced the long-run behaviour from different states.

Both simulations were performed using 1992 as the seed for the random-number generation, so results were reproducible.

Simulation	Starting state
1	Empty state - 00000
2	Full state - 20010

Table 8: Simulation - starting points.

Algorithm 3.8.

1. Find the position of the starting state through function **find**. Initialise simulation counters. Visit-state, State-history, Accepted-arrival, Rejected-arrival, Observed-distance.
2. At each beat, obtain the current state and generate its *shifted position* as per 2.1 (See B.4, lines 1-5).
3. Generate a random number **r** using **rand**, where $r \sim Uniform(0, 1)$, meaning **r** follows a **Uniform distribution**, hence, every number is equally likely to be selected (See B.4, line 6).

The random *event* applied in the following *beat*, is given by the value of **r**, according to the criteria (See B.4, lines 7-29):

$$r = \begin{cases} 0 \leq r < p_n, & \text{no arrival} \\ p_n \leq r < p_n + p_s & \text{arrival of e190} \\ p_n + p_s \leq r \leq 1 & \text{arrival of 737} \end{cases} \quad (3.7)$$

Each probability value is given by Table 12.

4. For e190 and 737 build the *candidate state* as per Section 3.5.2, step 3.5 (See B.4, lines 11,21).
5. Decide whether to accept or reject the *candidate state*, according to the criteria in equation 3.6.
 - (a) If **accepted**, the *candidate-state* becomes the *following-state* and *accepted-arrival-counter* increases by 1 (See B.4, line 12-14 and 22-24).
 - (b) If **rejected**, the *shifted-state* becomes the *following-state*, and *rejected-arrival-counter* increases by 1 (See B.4, lines 15-17 and 25-27).
6. Find the *next state* position using **find**, register the index in *state-history* and increase *visit-state* by 1 (See B.4, lines 30-32).
7. Register the observed-distribution as shown in Equation 2.13 (See B.4, line 33).
8. Compare the observed-distribution-counter versus stationary distribution using RSS, according to Equation 2.12 (See B.4, line 34).

Simulation counters obtained are:

1. Visit-state-counter: Number of times each state was visited.
2. State-history-counter: Sequence of states occupied per beat.
3. *Accepted – arrival – counter_{aircraft}*: Count arrivals accepted in the system per aircraft.
4. *Rejected – arrival – counter_{aircraft}*: Count arrivals that were rejected by the system per aircraft.
5. Observed-distribution-counter: Stores RSS per beat.

From the simulation counters, quantities were computed (See B.4.1, lines 1-18):

$$\text{Total arrivals}_{aircraft} = \text{accepted-arrival-counter}_{aircraft} + \text{rejected-arrival counter}_{aircraft} \quad (3.8)$$

$$\text{Rejection rate}_{aircraft} = \frac{\text{rejected-arrival-counter}_{aircraft}}{\text{total arrivals}_{aircraft}} \times 100 \quad (3.9)$$

Where *aircraft* represent e190 or 737, respectively. Similar calculations were applied to the combined counters to obtain the total arrivals and rejection rates.

3.5.6 Economic layer

The economic layer estimates the arrival-fee revenue and staff costs per turnaround per aircraft. Other expenses such as infrastructure, utilities, staff penalty rates are outside the scope of the model.

Staff cost.

The **hourly cost** ($\text{Cost}_{\text{hour}}$) estimation is:

$$\text{Cost}_{\text{hour}} = \sum_{\text{type}=1}^3 \text{Staff number}_{\text{type}} \times \text{Hourly rate}_{\text{type}}, \quad \text{type} \in \{1, 2, 3\} \quad (3.10)$$

With,

1. Staff number_{type}: Number of employees under certain level of classification. For staff composition see Appendix F.
2. Hourly rate_{type}: Value per hour given by employee classification, obtained from [8, p.8-10].

The **cost per beat** ($\text{Cost}_{\text{beat}}$) is given by:

$$\text{Cost}_{\text{beat}} = \text{Cost}_{\text{hour}} \times \frac{\text{beat (in minutes)}}{\text{minutes per hour}} \quad (3.11)$$

With,

1. *beat*: 6 minutes.
2. minutes per hour: 60

Hence, the **staff cost** per serviced aircraft ($\text{Cost}_{\text{aircraft}}$) is:

$$\text{Cost}_{\text{aircraft}} = \text{Cost}_{\text{beat}} \times \text{Service beats}_{\text{aircraft}} \quad (3.12)$$

The beat duration and service time given by Table 7.

Arrival revenue. Estimated as follows:

$$\text{Revenue}_{\text{aircraft}} = (\text{Arrival fee} \times \text{Capacity}_{\text{aircraft}}) \times \text{Occupancy}_{\text{aircraft}} \quad (3.13)$$

With,

1. Revenue_{aircraft}: Estimated revenue per aircraft type.
2. Arrival fee: 14.71 AUD per each passenger that lands in Adelaide Airport. Given by Adelaide airport [2, p.1].
3. Capacity_{aircraft}: From Table 5.
4. Occupancy_{aircraft}: Assumed 1.

Net revenue. Estimated as follows:

$$\text{Net revenue}_{\text{aircraft}} = \text{Revenue}_{\text{aircraft}} - \text{Cost} \quad (3.14)$$

Using the simulation counters defined in 3.5.5: *accepted-arrival-counter* and *rejected-arrival-counter* per aircraft. Net revenue can be subdivided into:

$$\text{Captured} = \sum_{\text{aircraft}} \text{Accepted-arrival-counter}_{\text{aircraft}} \times \text{Net revenue}_{\text{aircraft}} \quad (3.15)$$

$$\text{Missed} = \sum_{\text{aircraft}} \text{Rejected-arrival-counter}_{\text{aircraft}} \times \text{Net revenue}_{\text{aircraft}} \quad (3.16)$$

$$\text{Potential} = \text{Captured} + \text{Missed} \quad (3.17)$$

with,

1. Captured: net revenue generated by aircraft that were accepted and serviced.
2. Missed: net revenue that was not captured since aircraft were rejected.
3. Potential: total net revenue that could have been generated if all arrived aircraft had been accepted and serviced.

3.5.7 Sensitivity analysis

A sensitivity analysis was performed to examine how changes in the arrival rates impact the system. The arrival probabilities proportions between aircraft were preserved.

Algorithm 3.9.

1. For each *state*, generate *shifted* and *candidate* states, capacity assessment and rejection procedures described in 3.6. Store *following-state* indices in separated vectors per event (See B.6, lines 1-15).
2. Define arrival-rates amplifiers from 0 to 2.5, increasing progressively 0.1. Initialise vectors to store results with **zeros** (See B.6, lines 16-23).
3. Per each event, create amplified probabilities, retaining only those that comply with

$$p_{\text{amplified}_s} + p_{\text{amplified}_l} \leq 1$$

(See B.6, lines 24-32).

4. Rebuild the transition matrix with the new probabilities, as per Section 3.6 (See B.6, lines 33-38).
5. Calculate the TSD of the new transition matrix as per 3.7 to each amplified matrix (See B.6, lines 39-43).
6. Compute the occupancy per state as Equation (3.6).

If:

$$|state| = capacity \rightarrow \text{arrival rejected},$$

add v_i to the *rejection probability accumulator* (See B.6, lines 44-50).

7. Store *amplified arrival probabilities* and their *theoretical rejection percentage* for each arrival-rate multiplier (See B.6, lines 51-55).
8. For each multiplier, transform the *theoretical rejection probability* into expected accepted and rejected arrivals over *total beats*. Apply the net-revenue from Equation (3.14) to compute the captured, missed and potential net revenue, defined in 3.14, per scenario:

$$\text{Expected-amplified}_{\text{aircraft}} = \text{total beats} \times \text{AAp-beat}_{\text{aircraft}} \quad (3.18)$$

$$\text{Rejected-amplified}_{\text{aircraft}} = \text{total beats} \times \text{AAp-beat}_{\text{aircraft}} \times \text{RAp} \quad (3.19)$$

$$\text{Accepted-amplified}_{\text{aircraft}} = \text{total beats} \times \text{AAp-beat}_{\text{aircraft}} \times (1 - \text{RAp}) \quad (3.20)$$

With,

- (a) $\text{Expected-amplified}_{\text{aircraft}}$: Theoretical number of arrivals during a day, per aircraft.
- (b) $\text{Rejected-amplified}_{\text{aircraft}}$: Theoretical number of rejected arrivals during a day, per aircraft.
- (c) $\text{Accepted-amplified}_{\text{aircraft}}$: Theoretical number of accepted arrivals during a day, per aircraft.
- (d) *total beats*: 240. From Equation (3.2).
- (e) RAp: Theoretical amplified rejection probability.
- (f) $\text{AAp-beat}_{\text{aircraft}}$: Theoretical amplified arrival probability per beat, per aircraft.

4 Results

Pieces of code are referenced for Part A in Appendix A. Part B references are in Appendix B and D.

The full code is available in OneDrive and GitHub as Part A: [PartA_Final_h3_equal.m](#) Part B: Model [2.Part 2_h_3_weighted.m](#), Data estimation (PYTHON) [1.BBDD conversor.ipynb](#) and [2.Arrival_analysis_v1.ipynb](#).

Landed	Cancelled	Other	Total
4,792	64	41	4,897

Table 9: Raw data by flight status.

Gate registered	Gate unregistered	Total
3,738	1,054	4,792

Table 10: Landed by information completion.

4.1 Data estimation

Code is available in Appendix D.

Total collected raw data by flight status (D.2.3, lines 15-18): From 9 only Landed records were kept. Then, were split between gate information (D.2.3, lines 1-2):

From 10 only entries with registered gate were kept. Then, they were split into aircraft types (See D.2.3, lines 7-8):

Categorised - 737	Categorised - e190	Uncategorised	Total
2,000	635	1,103	3,738

Table 11: Arrivals by aircraft type.

From 11 Uncategorised records were removed. Therefore, replacing Table 11 values in equation (3.4) (See D.2.4, lines 5-7):

Event type	Probability (per beat)
p_s	0.0519
p_l	0.1634
p_n	0.7847

Table 12: Event probabilities per beat.

These arrival probabilities will later be used as an input to build our model's transition matrix.

4.2 State Space

From Part A, according to Equation (2.3), from code A.1.1, lines 1-7, 8 states were generated. Thus a graph for $J(0:3,3)$ is as follows:

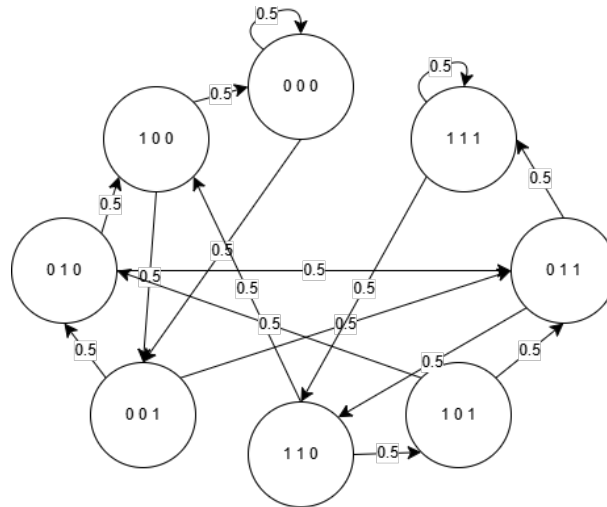


Figure 3: $J(0:3,3)$ directed graph generated in <https://app.diagrams.net/>.

Similarly, in Part B, 56 **candidate states** were generated by the loop (B.1.1, lines 2-15). After reducing them to those reachable **from the empty state**, 39 **valid states** remain (See B.1.2, lines 3-26). Since at most one movement is allowed per beat, there were 17 unreachable states that were eliminated (See B.1.2, line 29).

4.3 Transition Matrix

From Part A, a 8×8 transition matrix was built (See A.2.1, lines 1-7), that was then verified resulting in all its rows summing up to 1 (See A.2.2, line 2).

In Part B, a 39×39 transition matrix was generated from the reachable states (See B.2.1, lines 2-26), all valid transitions originated by the 3 events were registered (See B.2.1, line 12), then was verified with all its rows sum up to 1 (See B.2.2, line 2).

Heatmap diagrams for both models were generated (See G.).

4.4 Theoretical Stationary Distribution and Simulation

(See B.3.1, lines 1-27 and B.4, lines 1-35).

From the long-run per-beat rates from Equation 3.2, total beats= 240 results in Table 13.

Expected	e190	737	Total
Arrivals	12.46	39.22	51.67
Serviced	12.23	38.51	50.74
Rejected	0.22	0.71	0.93

Table 13: Long-run expected aircraft per model per day.

Theoretical versus simulated rejection rates per plane:

Aircraft	Theoretical Rejection (%)	Simulation 1 - Empty	Simulation 2 - Full
e190	1.80	1.77	1.84
737	1.80	1.73	1.82
Total	1.80	1.74	1.82

Table 14: Theoretical versus simulated rejection rates per aircraft.

Chosen Eigenvalues in both model were unique and equal to 1. Eigenvalues were plotted for both models in Appendix I.

4.4.1 Distribution

The theoretical stationary distribution was generated and verified (Appendix B.3.1, lines 2-12 and 15-24) The stationary distribution is illustrated in Figure 5, that was generated by code in C.3.2 lines 7-16, as follows:

From 4 all states are equally likely in the long-run. Theoretical and simulated visited frequencies are almost equal.

Figure 5 illustrates how in the long-term the distribution is uneven between states, where the system spends 34.03% of its time empty. Theoretical and simulated frequencies coincide for all states, regardless of the initial state. In addition, from Table 15, demonstrates that the system spends 80.05% of its time in only 8 states.

From another perspective, Figures 6 and 7, generated by code in Appendix C.3.2 lines 7 to 12, as follows:

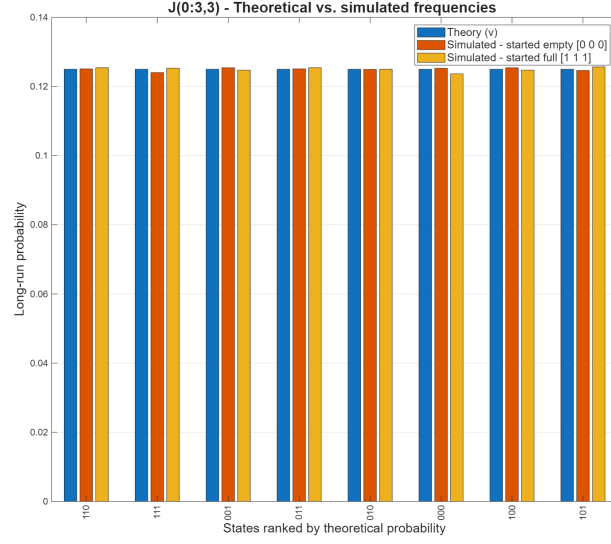


Figure 4: Part A - J(0:3,3) Visited States in descendant order.

Rank	State	Theoretical distribution(%)	Cumulative distribution(%)
1	00000	34.0278	34.0278
2	00100	9.3201	43.3479
3	10000	8.8970	52.2449
4	01000	8.8834	61.1283
5	00001	7.0857	68.2140
6	00010	7.0857	75.2997
7	01100	2.4195	77.7192
8	11000	2.3293	80.0485

Table 15: Theoretical stationary distribution - top 8

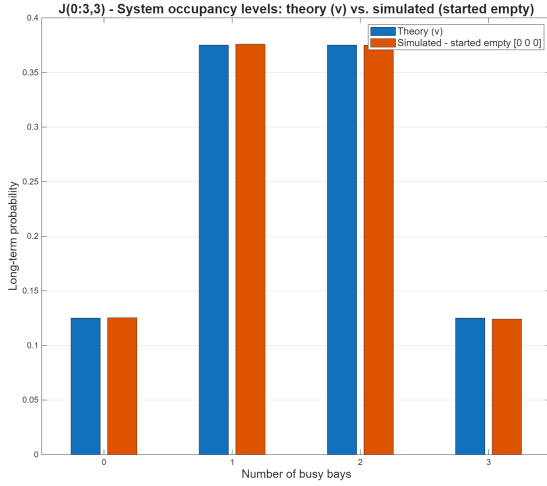


Figure 6: Part A - J(0:3,3) System occupancy levels per number of busy bays.

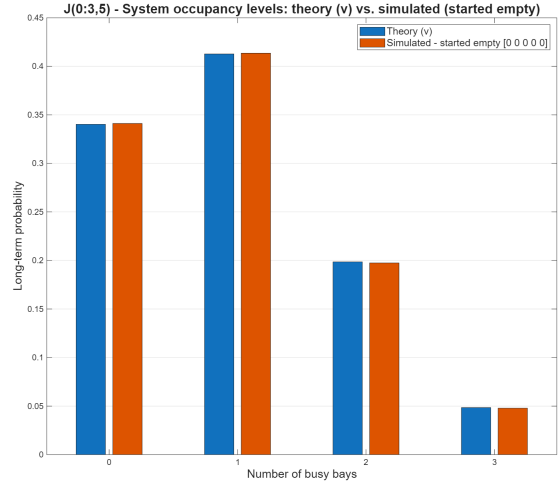


Figure 7: Part B - J(0:3,5) System occupancy levels per number of busy bays.

In part A, the bay occupancy level is symmetric among the system states, as seen in Figure 6, which is confirmed by the mean number of occupied bays is 1.5, with a long-run utilisation of 50.00% (See code C.3.4, lines 1-2).

In Part B Figure 7 the system stays empty a 34.03% of its time, with one busy bay 41.27%, two bays 19.86% and completely busy 4.85%. The mean number of occupied bays is 0.96, with a long-run utilisation of 31.84% (See code C.3.4, lines 1-2).

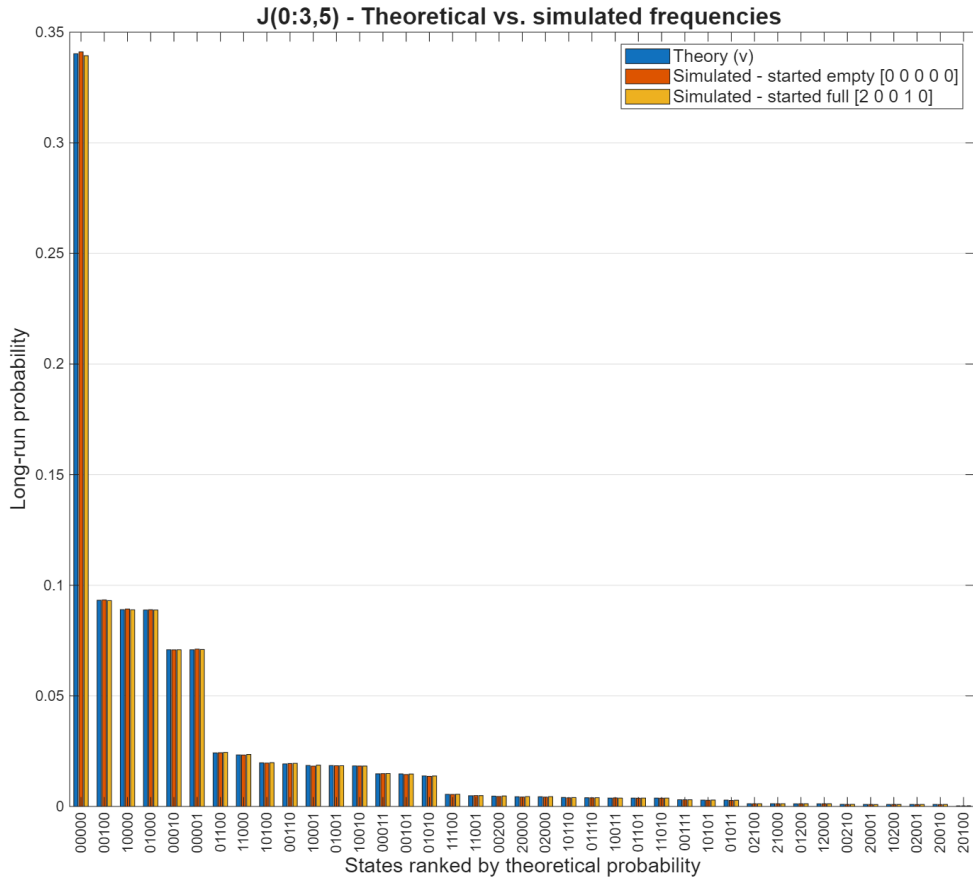


Figure 5: Part B - $J(0:3,5)$ Visited States in descendant order.

4.4.2 Convergence

From Figures 8 and 9, it can be seen how RSS decreases its values as steps increase. At around 1,000 steps, the simulations tend to converge, reaching similar distance magnitudes, regardless of their starting point. Plots were generated by code in Appendix C.2.1. Note that logarithmic function was applied to both axis to provide a clearer representation of the convergence.

Additionally, Table 16 show RSS at four simulated steps for Part A:

Step	Empty Start	Full Start
10	0.0750	0.0950
1,000	0.000222	0.0001760
100,000	0.000009792	0.000006341
1,000,000	0.000001521	0.000002846

Table 16: RSS values at selected simulation steps for Part A

Tables 16 and 17 were generated by code C.2.2.

Similarly, Table 17 show the RSS at four simulated steps for Part B:

Step	Empty Start	Full Start
10	0.03908	0.1996
1,000	0.0006444	0.0003527
100,000	0.000003572	0.00001544
1,000,000	0.000001311	0.000001098

Table 17: RSS values at selected simulation steps for Part B

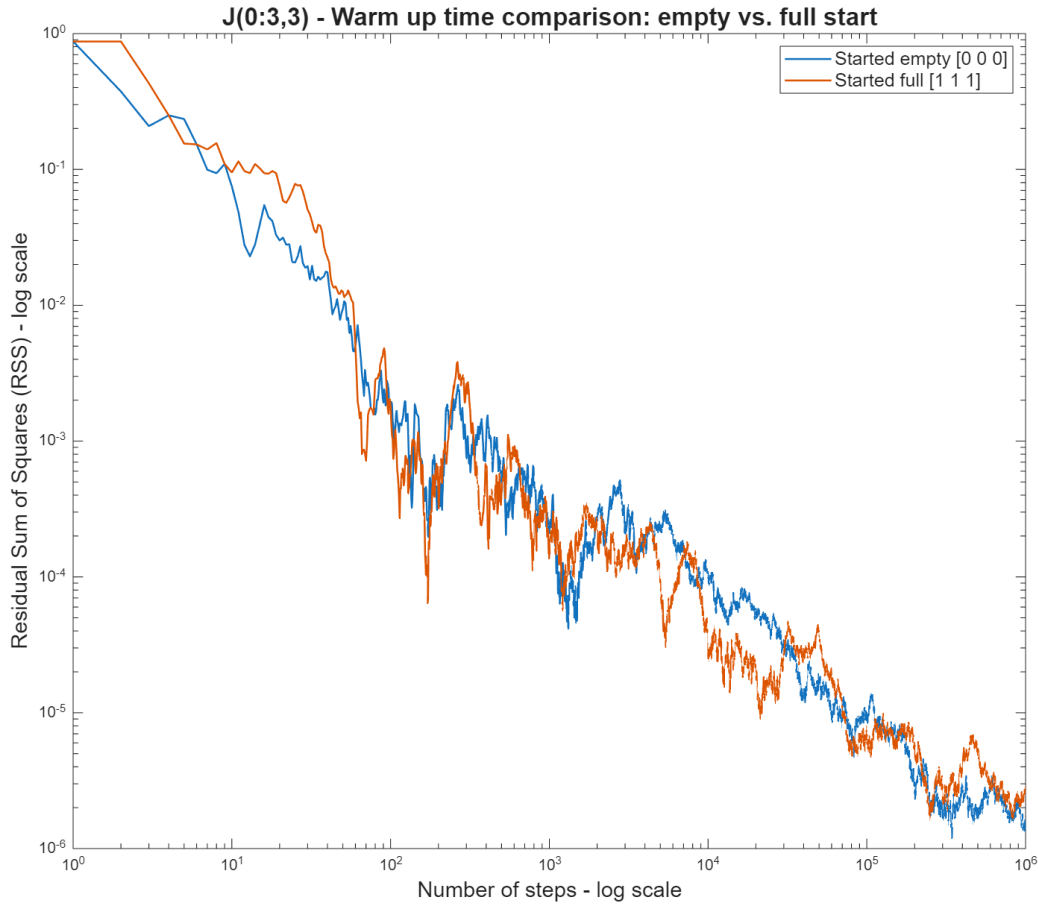


Figure 8: Part A - $J(0:3,3)$ Warm-up time observed between simulations and theoretical distribution.

4.5 Economic layer

From an economic perspective, using Equation 3.14 the following values can be obtained:

Aircraft	Service Beats	Revenue (AUD)	Staff cost (AUD)	Net revenue (AUD)	Net revenue per beat (AUD)
e190	3	1,323.900	119.61	1,204.29	401.43
737	5	2,471.28	199.36	2,271.93	454.39

Table 18: Cost and revenue per aircraft.

Replacing values from Table 13 and Table 18, in Equations 3.15, 3.16 and 3.17 the results from Table 19 were obtained:

Net revenue (AUD)	e190	737	Total
Captured	14,730.36	87,490.67	102,221.03
Missed	270.25	1,605.13	1,875.38
Potential	15,000.61	89,095.80	104,096.41

Table 19: Long-run net revenue per model per day.

Rates are equal by construction rather than coincidence: an arrival is accepted whenever the shifted state capacity allows it, since both aircraft occupy one bay, acceptance does not **depend on types**. Congestion then, rejects aircraft at a 1.80% rate. But since 737 arrive 3 times more often, 0.71 are rejected per model day against 0.22 e190s, then 737 misses 2,271.93 AUD of net revenue against 1,204.29 AUD for an e190.

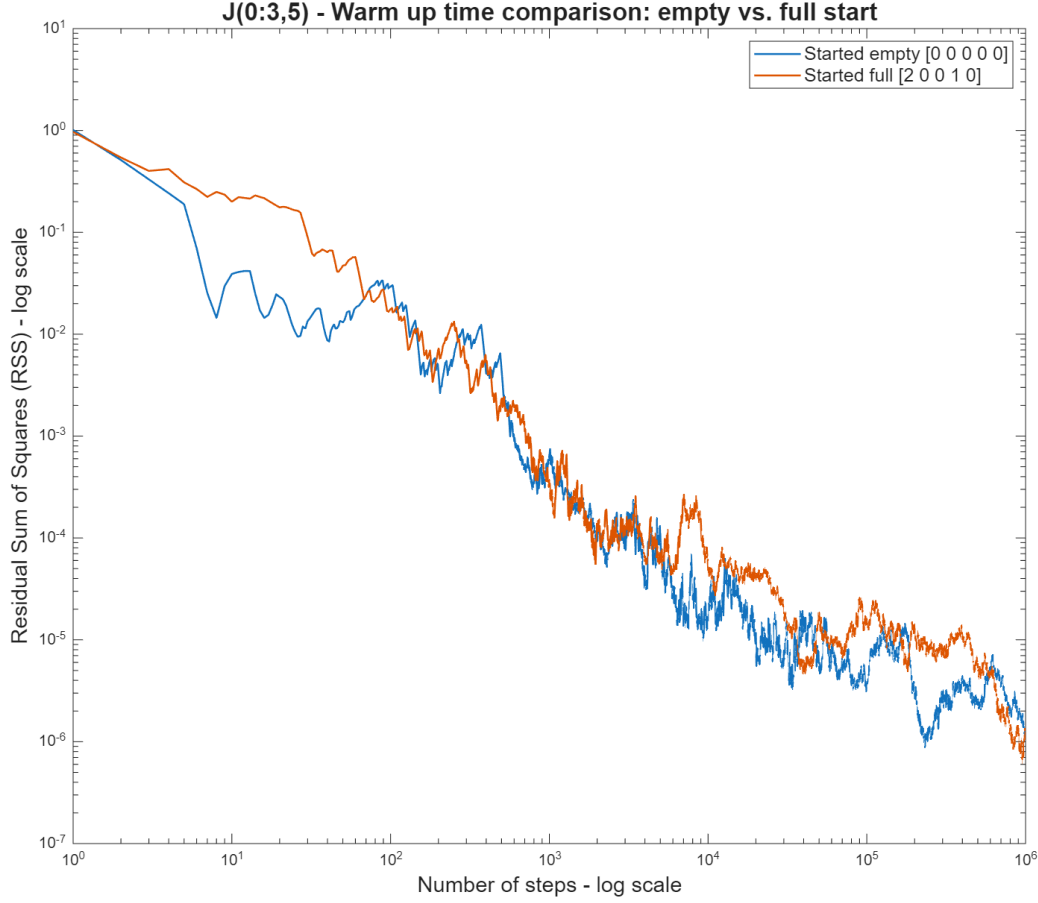


Figure 9: Part B - J(0:5,3) Warm-up time observed between simulations and theoretical distribution.

4.6 What if arrival rates were higher?

From Code C.4 the following plot was built:

Applying Equations (3.18) and (3.20) with $t = 240$ beats, the expected arrivals, services and rejections were converted into net revenue for each multiplier Table 20.

Amplification factor	Rejection (%)	Captured (AUD per day)	Missed (AUD per day)	Potential (AUD per day)
1.0	1.80	102,221	1,875	104,096
1.5	4.88	148,527	7,618	156,145
2.0	9.20	189,044	19,149	208,193
2.5	14.25	223,164	37,077	260,241

Table 20: Economic effects of amplifying arrival rates.

Part A has a rejection rate of 0.00%. Unlike Part B that from Table 14 and Figure 10, based on the current arrival rate 1.0 amplification factor, has a rejection probability of 1.80% that increases strongly when amplifying the arrival rate, reaching up to 14.25% when amplification factor is at 2.5.

From Code C.5 the following plot was built:

From Figure 11 and Table 20, at twice the arrival rate, captured revenue reaches 189,043.91 AUD per day and missed net revenue 19,149 AUD per day.

5 Discussion

Part A serves as a methodology validation. Since $h = \text{capacity} = \text{service duration} = 3$, binary entries and at most one arrival per beat. After shifting, the final position became available so, every new arrival was accepted and the rejection rate was 0.00%. The stationary distribution was uniform since all 8 states have 12.50% probabilities of being occupied.

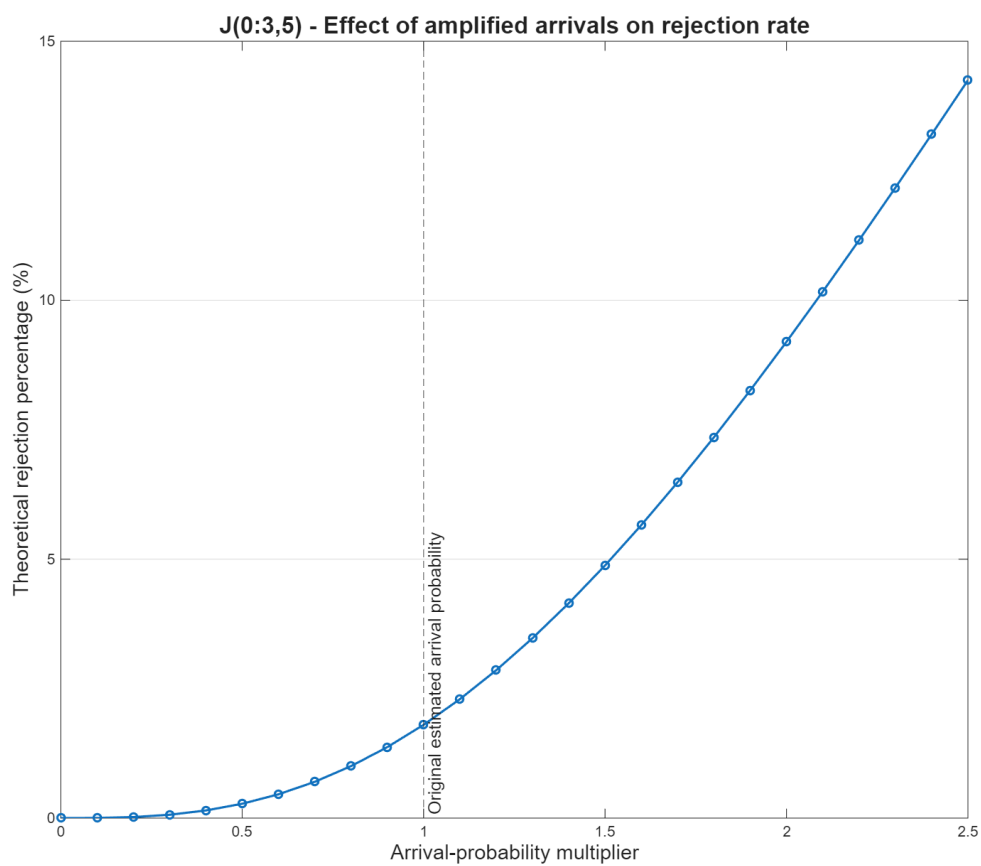


Figure 10: Effect of amplifying arrival over rejection rates

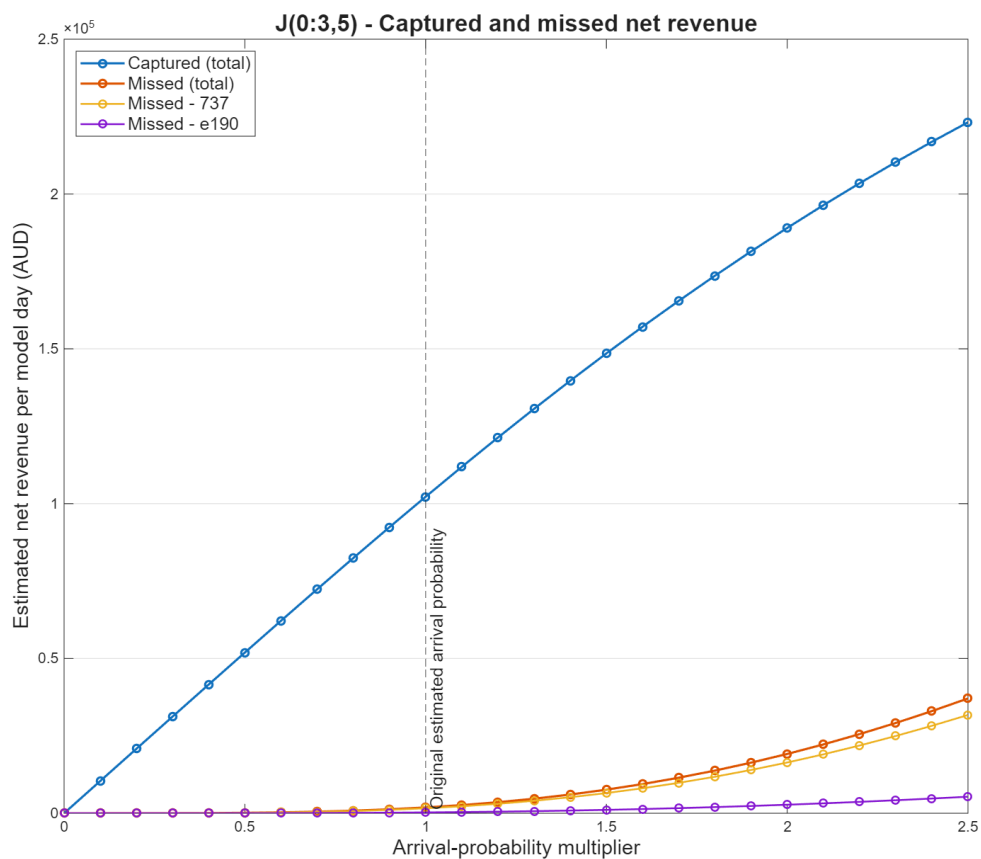


Figure 11: Total captured and missed revenue per plane.

From Part B long-term probabilities are concentrated primarily among low-occupancy states with a rejection rate of 1.80% is not unexpected, as the system is full only 4.85% of its time, which is the only period when rejections may occur. However, this only happens when the shifted state was full after shifting. To examine the rejection rate behaviour, arrival rates were amplified, resulting in rejection rate increasing from 1.80% up to 9.20%, augmenting in 5.1 times its original value when arrival rates were doubled.

Comparing both models gives deeper insight into the causes of congestion **in the long-run**. Part A operates at a higher arrival rate per beat of 0.50, and a higher mean bay occupancy of 1.5, thus 50.00% of system utilisation; however, it has a 0.00% rejection rate. While Part B rejects 1.80% of its arrivals, with an arrival rate per beat of 0.2153, a mean occupied bays of 0.96 and a system utilisation of 31.84%.

Rejection rates of Part A can be understood if we look at the assumptions of the model, its capacity will never be exceeded by its arrivals under its structure, since the *shifting mechanism* will always free a bay that may be occupied by another arrival in the next beat. On the other hand, Part B may reject arrivals since its horizon of time leaves a gap to arrivals happening when the system has already reached its capacity; that may continue rejecting aircraft until the current serviced aircraft finish. Hence, it is suggested that congestion is strongly influenced by the structure of the modelled system: its horizon of time, capacity, service time and accepted aircraft per beat, not only by arrival rates alone.

Saying the above, and thinking of the number of occupied bays as an abstract representation of the system energy, the stationary distribution suggests, when given unequal arrival rates, the system tends to settle towards lower levels of energy.

Convergence within roughly 1,000 beats, equivalent to about four days of continuous operation, suggests that the long-run results are **not sensitive** to the initial state where simulations started.

Since both aircraft occupy one bay, the system rejects them at the same rate; what differs is how much each of them **contributes per beat** of bay time. An e190 contributes 401.43 AUD per beat against 454.39 AUD for a 737, hence the latter returns approximately 13.2% during the same stay. This suggests that, when the system is full, giving priority to larger aircraft would increase the captured net revenue without adding an extra bay.

Acknowledgements

- The link to the GitHub repository where the information is published is as follows:
<https://github.com/chenriquezve/PR-B>
- I would like to thank my supervisor for his constant support, guidance, and patience during this project.
- I acknowledge the use of QuillBot for language refinement and grammatical support during the writing process.

References

- [1] Abd Allah Makhloof, M., Elsayed Waheed, M., & El Raouf Badawi, U. A. (2014). *Real-time aircraft turnaround operations manager*. Production Planning & Control, 25(1), 2–25. <https://doi.org/10.1080/09537287.2012.655800>
- [2] Adelaide Airport Limited. (2026). *Schedule of aeronautical fees*. <https://corporate.adelaideairport.com.au/media/hddjuqwd/aal-schedule-of-fees-20260701.pdf>
- [3] Buhler, J., Eisenbud, D., Graham, R., & Wright, C. (1994). *Juggling Drops and Descents*. The American Mathematical Monthly, 101(6), 507. <https://doi.org/10.2307/2975316>
- [4] Cardinal, J., Kremer, S., & Langerman, S. (2006). *Juggling with Pattern Matching*. Theory of Computing Systems, 39(3), 425–437. <https://doi.org/10.1007/s00224-005-1239-x>
- [5] Polster, B. (2003). *The mathematics of juggling*. Springer-Verlag. <https://doi.org/10.1007/b98883>
- [6] Warrington, G. S. (2005). *Juggling Probabilities*. The American Mathematical Monthly, 112(2), 105. <https://doi.org/10.2307/30037409>
- [7] Warrington, G. S. (2008). *Juggling Performers + Math = ?* Math Horizons, 15(3), 18–21. <https://doi.org/10.1080/10724117.2008.11974754>

- [8] Fair Work Ombudsman. (2026). *Pay guide—Airport Employees Award [MA000049]*. <https://portal.fairwork.gov.au/ArticleDocuments/872/airport-employees-award-ma000049-pay-guide.pdf.aspx>
- [9] Gagniuc, P. A. (2017). *Markov Chains: From Theory to Implementation and Experimentation* (1st ed.). John Wiley et Sons Ltd.
- [10] Grimmett, G., & Stirzaker, D. (2001). *Probability and random processes* (3rd ed.). Oxford University Press.
- [11] Stewart, W. J. (2009). *Probability, Markov Chains, Queues, and Simulation The Mathematical Basis of Performance Modeling* (1st ed.). Princeton University Press. <https://doi.org/10.1515/9781400832811>
- [12] Schmidt, M. (2017). *A review of aircraft turnaround operations and simulations*. Progress in Aerospace Sciences, 92, 25–38. <https://doi.org/10.1016/j.paerosci.2017.05.002>
- [13] Qantas. *Boeing 737-800*. Aircraft seat map. <https://www.qantas.com/adobe/assets/urn:aaid:aem:d3287260-d46c-4a38-9d3f-80cf696d283c/original/as/boeing-737-800-seat-map.pdf>
- [14] Qantas. *Embraer E190 seat map: Configuration 1*. Aircraft seat map. <https://www.qantas.com/adobe/assets/urn:aaid:aem:4cb66991-63a6-4087-9f4c-2b9af7229d7b/original/as/e190-seat-map-config-1.pdf>
- [15] Norris, J. R. (1997). *Markov Chains* (1st ed., Vol. 2). Cambridge University Press. <https://doi.org/10.1017/CBO9780511810633>
- [16] Adelaide Airport Limited. (2026). *Connecting South Australia: Aviation update, February 2026*. <https://corporate.adelaideairport.com.au/media-centre/?page=2&pageSize=12&search=&year=2026>
- [17] Wu, C. L., & Caves, R. E. (2004). *Modelling and optimization of aircraft turnaround time at an airport*. Transportation Planning and Technology, 27(1), 47–66. <https://doi.org/10.1080/0308106042000184454>
- [18] Schultz, M., Evler, J., Asadi, E., Preis, H., Fricke, H., & Wu, C. L. (2020). *Future aircraft turnaround operations considering post-pandemic requirements*. Journal of Air Transport Management, 89, Article 101886. <https://doi.org/10.1016/j.jairtraman.2020.101886>

Appendices

A Code quotes - Model Part A

A.1 State Space Construction

Full code of Part A is available in [PartA_Final_h3_equal.m](#) provided in OneDrive and GitHub.)

A.1.1 States generation

From lines 8 to 14 the code print the states that will be formed. Lines 16 to the end of the code store each of them. The output is saving a vector per b.

```
1 for b = 0:h
2     all_states = nchoosek(1:h, b);
3     n_states = size(all_states, 1);
4     state_vals = zeros(n_states, h);
5     for i = 1:n_states
6         state_vals(i, all_states(i,:)) = 1;
7     end
8     fprintf('From h = %d and b= %d: \n', h, b)
9     fprintf('    All possible states (or edges): \n');
10    disp(all_states);
11    fprintf('    Number of states (or edges): \n');
12    disp(n_states);
13    fprintf('    All states (or edges) values: \n');
14    disp(state_vals);
15    if b == 0 %Saving one file per b (b goes from 0 to h)
16        all_states_b0 = nchoosek(1:h, b);
17        n_states_b0 = size(all_states_b0, 1);
18        state_vals_b0 = zeros(n_states_b0, h);
19        for i = 1:n_states_b0
20            state_vals_b0(i, all_states_b0(i,:)) = 1;
21        end
```

The code was deliberately cut in line 21, since the process for each remaining b (b=1, b=2, b=3) is the same as shown in lines 15-21.

A.1.2 States merged

The row vectors generated in A.1.1, are merged as a matrix to use it later. The output is saving a matrix of valid states.

```
1 state_vals_merged = [state_vals_b0; state_vals_b1;
2     state_vals_b2; state_vals_b3];
3 %Checking matrix dimensions:
4 n_states_merged = size(state_vals_merged, 1);
5 fprintf('State values merged: \n');
6 disp(state_vals_merged);
```

A.2 Transition matrix

A.2.1 Construction

Manually a transition matrix was built, by hand.

```
1 All_transitions_merged = zeros(n_states_merged, n_states_merged);
2 %State 1 -> [0 0 0] to:
3     All_transitions_merged(1,1) = 0.5; %State 1 -> [0 0 0]
4     All_transitions_merged(1,4) = 0.5; %State 4 -> [0 0 1]
5 %State 2 -> [1 0 0] to:
6     All_transitions_merged(2,1) = 0.5; %State 1 -> [0 0 0]
```



```
7 All_transitions_merged(2,4) = 0.5; %State 4 -> [0 0 1]
```

The code was deliberately cut in line 7, since the same process showed in lines 3-4 or 5-6, is repeated in every remaining state.

A.2.2 Verification

Line 2 verifies the built transition matrix is valid.

```
1 fprintf('Row sums (should all be , otherwise check that row's probs:\n')
2 disp(sum(All_transitions_merged , 2))
```

A.3 Stationary Distribution

The code applied to compute the stationary distribution from Part A is exactly the same as the one used for Part B. Please refer to Appendix B, Code B.3.1.

A.4 Simulation

2 simulations of 1,000,000 steps each, were performed. A seed was used to generate the random number in line 5, for replicability purposes as [rng\(1992\)](#), located at the beginning of the code. The Output for this code are 2 simulations performed, with 5 counters generated (see lines 19-23). A definition per each is available in [3.5.5](#).

```
1 for time = 1:steps
2     prev_state = curr_state;
3     current_vec = valid_states(prev_state, :);
4     shifted_vec = [current_vec(2:end), 0]; %Depart + shift
5     r = rand;
6     if r < p_beat_non_arrival %Event 1: No arrival
7         next_vec = shifted_vec;
8     elseif r < p_beat_non_arrival + p_beat %Event 2: arrival
9         candidate_vec = shifted_vec;
10        candidate_vec(service_beats) = candidate_vec(service_beats) + 1;
11        if sum(candidate_vec) <= capacity
12            next_vec = candidate_vec; %Seated
13            simulation_counter1 = simulation_counter1 + 1;
14        else
15            next_vec = shifted_vec; %Rejected
16            rejected_counter1 = rejected_counter1 + 1;
17        end
18    end
19    curr_state = find(all(valid_states == next_vec, 2), 1);
20    state_history1(time) = curr_state;
21    visit_count(curr_state) = visit_count(curr_state) + 1;
22    observed_distr = visit_count/time;
23    rss_time_state1(time) = sum ((observed_distr - v').^2);
24 end
```

B Code quotes - Model Part B

This appendix contains pieces of code that were processed using **MATLAB**. However, the full code of Part B is available in [2. Part2_h_3_weighted.m](#) in OneDrive and GitHub.

B.1 State Space Construction

A *valid states* list is created in line 1.

From lines 2-16 all vectors complying with $h = 5, b = 3, capacity = 3$ are being generated and stored in *valid states* list.

Line 16 print the size of *valid states* list, giving the number of results contained, which are referred to as *possible states* in line 17.

B.1.1 States generation

```

1 valid_states = [];
2 for p1 = 0:b
3     for p2 = 0:b
4         for p3 = 0:b
5             for p4 = 0:b
6                 for p5 = 0:b
7                     candidate_state = [p1 p2 p3 p4 p5];
8                     if sum(candidate_state) <= capacity
9                         valid_states(end+1, :) = candidate_state;
10                    end
11                end
12            end
13        end
14    end
15 end
16 n_states_raw = size(valid_states, 1);
17 fprintf('Number of possible states: %d\n', n_states_raw);

```

B.1.2 Trim to reachable states

Lines 3-8 check every state in list *valid state* shifting it and adding each event separately, the aim of this code is to keep only the states that are reachable **from the empty state 00000**, under the assumptions and restrictions stated in Methods Section.

Parameters:

state_to_check = 1

e190_service.beat=3

737_service.beat=5

```

1 possible_states = valid_states;
2 reachable_states = zeros(1, h); %Starting from 'empty state'
3 while state_to_check <= size(reachable_states, 1)
4     current_state = reachable_states(state_to_check, :);
5     shifted_state = [current_state(2:end), 0]; %Shifting state
6     possible_next_states = shifted_state; %no arrival
7     candidate_state = shifted_state; %e190 arrival
8     candidate_state(e190_service_beats) = candidate_state(
9         e190_service_beats) + 1;
10    if sum(candidate_state) <= capacity
11        possible_next_states(end+1, :) = candidate_state;
12    end
13    candidate_state = shifted_state; %737 arrival
14    candidate_state(b737_service_beats) = candidate_state(
15        b737_service_beats) + 1;
16    if sum(candidate_state) <= capacity
17        possible_next_states(end+1, :) = candidate_state;
18    end
19    for next = 1:size(possible_next_states, 1) %possible next states
20        candidate_state = possible_next_states(next,:);
21        is_possible=ismember(candidate_state, possible_states, 'rows');
22        reached_OK = ismember(candidate_state, reachable_states, 'rows');
23        if is_possible && ~reached_OK
24            reachable_states(end+1, :) = candidate_state;
25        end
26    end
27 end

```

```

25     state_to_check = state_to_check + 1; %move to the next state
26 end
27 valid_states = reachable_states;
28 n_states = size(valid_states, 1);
29 discarded_states = n_states_raw - n_states;
30 fprintf('Number of reachable states: %d. Hence, %d states were discarded
    .\n', n_states, discarded_states);

```

B.2 Transition Matrix: Values and verification

A Transition Matrix was created registering every probability of transition $p_{i,j}$ where valid transitions, that comply with the model assumptions and limitations, take place.

B.2.1 Transition matrix creation

```

1 [P_trans_matrix = zeros(n_states, n_states);
2 for current_state_number = 1:n_states
3     current_state = valid_states(current_state_number, :);
4     shifted_state = [current_state(2:end), 0]; %Shift, then append 0
5 %---Event 1: No arrival
6     next_state = shifted_state;
7     next_state_number = find(all(valid_states == next_state, 2)); %
    Finding state
8     if isempty(next_state_number)
9         fprintf('No arrival state was not found:\n')
10        disp(next_state)
11    end
12    P_trans_matrix(current_state_number, next_state_number) =
        P_trans_matrix(current_state_number, next_state_number) +
        p_beat_non_arrival); %Adding 'p_non_arrival' to 'P'
13 %---Event 2: e190 - Can an e190 be added?
14     candidate_state = shifted_state;
15     candidate_state(e190_service_beats) = candidate_state(
        e190_service_beats) + 1;
16     if sum(candidate_state) <= capacity
17         next_state = candidate_state; %Accepted
18     else
19         next_state = shifted_state; %Rejected
20     end
21     next_state_number = find(all(valid_states == next_state, 2)); %
    Finding state
22     if isempty(next_state_number)
23         fprintf('Following e190 state was not found:\n')
24         disp(next_state)
25     end
26     P_trans_matrix(current_state_number, next_state_number) = (
        P_trans_matrix (current_state_number, next_state_number) +
        p_beat_e190); %Adding 'p_beat_e190' to 'P'

```

The code was deliberately cut in line 27, since the same process showed in lines 14 to 27 took place, but analysing the case of a 737 arrival.

B.2.2 Transition matrix verification

The transition matrix created in the piece of code from above, was verified in this subsection, specifically in line 2. All its rows must sum up to 1, otherwise, the matrix is not a Markovian matrix.

```

1 fprintf('Row sums (should all be 1, otherwise check that row):\n')
2 disp(sum(P_trans_matrix, 2).')

```

B.3 Theoretical Stationary Distribution

Theoretical stationary distribution was computed from using $tolerance = 0.00001$, the line by line is described in 3.5.4

B.3.1 Main code

```
1 [eig_vector, eig_value] = eig (P_trans_matrix');
2 eigenvalues = diag(eig_value); %extracting eigenvalue
3 fprintf('Eigenvalues:\n')
4 disp(diag(eig_value'))
5 column_eig = find(abs(eigenvalues - 1) < tolerance);
6 fprintf('Number of eigenvalues = 1: %d\n', length(column_eig))
7 if length(column_eig) ~= 1
8     fprintf('There are %d eigenv. = 1!', length(column_eig))
9 end
10 fprintf('Selected eigenvector column: %d\n', column_eig)
11 v = eig_vector(:, column_eig); % extracting eigenvector
12 v = v / sum(v); %Normalising the eigenvector (so the probs add to 1)
13 %Verifying that result v is stationary (correct):
14 fprintf(Double checking (v * P = v))
15 v_step = v' * P_trans_matrix;
16 % Difference between v and v*P (should be close to zero)
17 diff_v = norm(v_step - v');
18 fprintf('    Difference (should be close to 0): %.10f\n', diff_v)
19 fprintf('    Used tolerance: %.4d\n', tolerance);
20 if diff_v < tolerance
21     fprintf('    -> v IS the stationary distribution\n')
22 else
23     fprintf('    -> v is NOT the stationary distribution, please
24         check it again\n')
25 end
26 fprintf('Theoretical stationary distribution:\n')
27 disp(v')
```

B.4 Simulation – Long-term distribution

2 simulations of 1,000,000 steps each, were performed. A seed was used to generate the random number in line 6, for replicability purposes as [rng\(1992\)](#), located at the beginning of the code. The Output for this code are 2 simulations performed, with 5 counters generated (see lines 30-35), a definition per each is available in 3.5.5.

Starting points were 00000 and 20010. This piece of code only shows the simulation for 00000; however, the simulation for 20010 follows the same steps and can be seen in the full code document.

```
1 curr_state = find(all(valid_states == [0 0 0 0 0], 2), 1)
2 for time = 1:steps
3     prev_state = curr_state;
4     current_vec = valid_states(prev_state, :);
5     shifted_vec = [current_vec(2:end), 0]; %Depart + shift
6     r = rand;
7     if r < p_beat_non_arrival %Event 1: No arrival
8         next_vec = shifted_vec;
9     elseif r < p_beat_non_arrival + p_beat_e190 % Event 2: e190
10         candidate_vec = shifted_vec;
11         candidate_vec(e190_service_beats) = candidate_vec(
12             e190_service_beats) + 1;
13         if sum(candidate_vec) <= capacity
14             next_vec = candidate_vec; %Seated
15             simulation_counter1_e190 = simulation_counter1_e190 + 1;
16         else
17             next_vec = shifted_vec; %Rejected
```

```

17         rejected_counter1_e190 = rejected_counter1_e190 + 1;
18     end
19     else % Event 3: 737
20         candidate_vec = shifted_vec;
21         candidate_vec(b737_service_beats) = candidate_vec(
            b737_service_beats) + 1;
22         if sum(candidate_vec) <= capacity
23             next_vec = candidate_vec; % Seated
24             simulation_counter1_737 = simulation_counter1_737 + 1;
25         else
26             next_vec = shifted_vec; %Rejected
27             rejected_counter1_737 = rejected_counter1_737 + 1;
28         end
29     end
30     curr_state = find(all(valid_states == next_vec, 2), 1);
31     state_history1(time) = curr_state;
32     visit_count(curr_state) = visit_count(curr_state) +1;
33     observed_distr = visit_count/time;
34     rss_time_state1(time) = sum ((observed_distr - v').^2);
35 end

```

B.4.1 Simulation calculations

Following equations were calculated inside the matlab document.

```

1 sim_seated_total1 = simulation_counter1_e190 + simulation_counter1_737;
2 sim_rejected_total1 = rejected_counter1_e190 + rejected_counter1_737;
3 sim_arrivals_total1 = sim_seated_total1 + sim_rejected_total1;
4 sim_rejection_rate1_e190=(rejected_counter1_e190/(
    simulation_counter1_e190+rejected_counter1_e190))*100;
5 sim_rejection_rate_total1=(sim_rejected_total1/sim_arrivals_total1)*100;
6 sim_arrivals1_e190 = simulation_counter1_e190 + rejected_counter1_e190;
7 sim_arr_beat1_e190 = sim_arrivals1_e190/steps;
8 sim_serv_beat1_e190 = simulation_counter1_e190/steps;
9 sim_rej_beat1_e190 = rejected_counter1_e190/steps;
10 sim_arr_beat1_total = sim_arrivals_total1/steps;
11 sim_serv_beat1_total = sim_seated_total1/steps;
12 sim_rej_beat1_total = sim_rejected_total1/steps;

```

Only individual counters for e190 were shown, since individual counters for 737 follow the same logic.

B.5 Economic layer

Parameters used:

```

n_ground_staff_level_2 = 8;
n_ground_staff_level_3 = 5;
n_engineer_level_3 = 1;
cost_ground_staff_level_2 = 26.95;
cost_ground_staff_level_3 = 27.54;
cost_engineer_level_3 = 45.41;
transit_arrival_fee = 14.71;
e190_seats_capacity = 90;
b737_seats_capacity = 168;
e190_occupancy_level = 1;
b737_occupancy_level = 1;

```

B.5.1 Service team cost

```
1 %Per hour:
2 estimated_cost_hour = (n_ground_staff_level_2 *
    cost_ground_staff_level_2) +(n_ground_staff_level_3 *
    cost_ground_staff_level_3) + (n_engineer_level_3 *
    cost_engineer_level_3);
3 %Per beat:
4 estimated_cost_beat = estimated_cost_hour * (beat/mins_per_hour);
5 %Per aircraft:
6 e190_estimated_team_cost_etc = estimated_cost_beat * e190_service_beats;
7 b737_estimated_team_cost_etc = estimated_cost_beat * b737_service_beats;
```

B.5.2 Arrival fees

```
1 e190_arrival_fee = e190_seats_capacity * e190_occupancy_level *
    transit_arrival_fee;
2 b737_arrival_fee = b737_seats_capacity*b737_occupancy_level*
    transit_arrival_fee;
```

B.5.3 Net revenue

```
1 e190_profit = e190_arrival_fee - e190_estimated_team_cost_etc;
2 b737_profit = b737_arrival_fee - b737_estimated_team_cost_etc;
```

B.6 Sensitivity analysis

```
1 for state = 1:n_states
2     current_vec = valid_states(state, :);
3     shifted_vec= [current_vec(2:end), 0];
4     next_N(state) = find(all(valid_states == shifted_vec, 2), 1);
5     candidate_vec = shifted_vec;
6     candidate_vec(e190_service_beats)=candidate_vec(e190_service_beats)
7         +1;
8     if sum(candidate_vec) > capacity, candidate_vec = shifted_vec;
9     end
10    next_S(state)=find(all(valid_states == candidate_vec,2), 1);
11    candidate_vec = shifted_vec;
12    candidate_vec(b737_service_beats)=candidate_vec(b737_service_beats)
13        +1;
14    if sum(candidate_vec) > capacity, candidate_vec = shifted_vec;
15    end
16    next_L(state) = find(all(valid_states == candidate_vec, 2), 1);
17 end
18 factor_min = 0.0;%minimum amplifier
19 factor_step= 0.1;%gap btween
20 factor_max = 2.5;%max tested
21 amplified =factor_min:factor_step:factor_max;
22 x_ampli_e190 = zeros(length(amplified), 1);
23 x_ampli_737= zeros(length(amplified), 1);
24 x_ampli = zeros(length(amplified), 1);
25 y_ampli = zeros(length(amplified), 1);
26 for m = 1:length(amplified)
27     p_e190_amplified = p_beat_e190*amplified(m); %amplified probs
28     p_737_amplified = p_beat_737*amplified(m);
29     p_n_amplified = 1 - p_e190_amplified - p_737_amplified;
30     if p_n_amplified < 0
31         x_ampli(m) = p_e190_amplified + p_737_amplified;
32         y_ampli(m) = NaN;
33         continue
34     end
35 end
```

```

32 end
33 P_trans_matrix_amplified = zeros(n_states, n_states);%rebuild P
34 for state = 1:n_states
35     P_trans_matrix_amplified(state, next_N(state)) =
        P_trans_matrix_amplified(state, next_N(state)) +
        p_n_amplified;
36     P_trans_matrix_amplified(state, next_S(state)) =
        P_trans_matrix_amplified(state, next_S(state)) +
        p_e190_amplified;
37     P_trans_matrix_amplified(state, next_L(state)) =
        P_trans_matrix_amplified(state, next_L(state)) +
        p_737_amplified;
38 end
39 [eig_vec_e190_ampli, eig_val_e190_ampli] = eig(
    P_trans_matrix_amplified');
40 eig_val_e190_ampli = diag(eig_val_e190_ampli);
41 column_eig_ampli = find(abs(eig_val_ampli - 1) < tolerance);
42 v_ampli = real(eig_vec_e190_ampli(:, col));
43 v_ampli = real(eig_vec_ampli(:, column_eig_ampli(1)));
44 v_ampli = v_ampli / sum(v_ampli);
45 reject_prob = 0;
46 for state = 1:n_states
47     occupancy_level_step = (valid_states(state,2)+valid_states(state
        ,3) + valid_states(state,4) + valid_states(state,5));
48     if occupancy_level_step >= capacity % reject
49         reject_prob = reject_prob + v_ampli(state);
50     end
51 end
52 x_ampli(m)= p_e190_amplified + p_737_amplified;
53 x_ampli_e190(m) = p_e190_amplified;
54 x_ampli_737(m)= p_737_amplified;
55 y_ampli(m)= reject_prob * 100;
56 end

```

C Plot Code Quotes

C.1 Stationary distribution

C.1.1 Eigenvalues graph

```

1 number_eigenvalues = length(eigenvalues);
2 eigenvalue_modulo = abs(eigenvalues);
3 figure;
4 eigenvalue_bars = bar(1:number_eigenvalues, eigenvalue_modulo);
5 xlabel('Eigenvector column', 'FontSize', 14);
6 ylabel('Absolute eigenvalue (|\lambda|)', 'FontSize', 14);
7 title('Absolute values of the transition matrix eigenvalues', '
    FontSize', 16);
8 xticks(1:number_eigenvalues);
9 ylim([0, 1]);

```

C.2 Simulation

C.2.1 Warm-up time

```

1 figure;
2 loglog(1:steps, rss_time_state1, 'LineWidth', 1.2); % empty start
3 hold on;
4 loglog(1:steps, rss_time_state2, 'LineWidth', 1.2); % full start
5 hold off;

```

```

6 xlabel('Number of steps - log scale', 'FontSize', 14);
7 ylabel('Residual Sum of Squares (RSS) - log scale', 'FontSize', 14);
8 title('J(0:3,5) - Warm up time comparison: empty vs. full start', '
  FontSize', 16);
9 legend('Started empty [0 0 0 0 0]', Started full [2 0 0 1 0], 'Location
  ', northeast , FontSize , 12);

```

C.2.2 RSS per points

```

1 checkpoints = [10, 1000, 100000, steps];
2 fprintf('Step\t\tEmpty start\t\tFull start\n');
3 for checkpoint = 1:length(checkpoints)
4     fprintf('%d\t\t%.6e\t\t%.6e\n', checkpoints(checkpoint),
5         rss_time_state1(checkpoints(checkpoint)), rss_time_state2(
6             checkpoints(checkpoint)));
7 end

```

C.3 Simulations vs. theory

C.3.1 Ranking states

To rank the states in descending order of frequencies, the following code was used:

```

1 [ranked_v, ranked_order] = sort(v, 'descend');
2 state_labels = join(string(valid_states), ', ', 2);
3 ranked_state_labels = state_labels(ranked_order);
4 ranked_freq_empty = freq_from_empty(ranked_order);
5 ranked_freq_full = freq_from_full(ranked_order);
6 cumulative_v = cumsum(ranked_v) * 100;
7 cumulative_empty = cumsum(ranked_freq_empty) * 100;
8 cumulative_full = cumsum(ranked_freq_full) * 100;

```

C.3.2 Plot visits per state

```

1 freq_from_empty = zeros(n_states, 1); %starting from [0 0 0 0 0]
2 freq_from_full = zeros(n_states, 1); %starting from [2 0 0 1 0]
3 %Normalising:
4 for state = 1:n_states
5     freq_from_empty(state) = sum(state_history1 == state) / steps;
6     freq_from_full(state) = sum(state_history2 == state) / steps;
7 end
8 state_labels = join(string(valid_states), ', ', 2);
9 comparison_values = [v, freq_from_empty, freq_from_full];
10 [ranked_v, ranked_order] = sort(v, 'descend');
11 ranked_state_labels = state_labels(ranked_order);
12 ranked_freq_empty = freq_from_empty(ranked_order);
13 ranked_freq_full = freq_from_full(ranked_order);
14 ranked_comparison_values = [ranked_v, ranked_freq_empty,
15     ranked_freq_full];
16 cumulative_v = cumsum(ranked_v) * 100;
17 cumulative_empty = cumsum(ranked_freq_empty) * 100;
18 cumulative_full = cumsum(ranked_freq_full) * 100;
19 %Ranking all states
20 figure;
21 bar(ranked_comparison_values);
22 xticks(1:n_states);
23 xticklabels(ranked_state_labels);
24 xtickangle(90);
25 xlabel('States ranked by theoretical probability', 'FontSize', 14);
26 ylabel('Long-run probability', 'FontSize', 14);

```



```

26 title(J(0:3,5) - Theoretical vs. simulated frequencies , FontSize ,
    16);
27 legend(Theory (v) , Simulated - started empty [0 0 0 0 0] , Simulated
    - started full [2 0 0 1 0] , Location , northeast , FontSize ,
    12);

```

C.3.3 Per number of busy bays (aggregated)

```

1 sim_occ = zeros(4, 1); %positions = 0,1,2,3 bays busy
2 for state = 1:n_states
3     occupancy_now = sum(valid_states(state,:));
4     sim_occ(occupancy_now + 1) = sim_occ(occupancy_now + 1) +
        freq_from_empty(state);
5 end
6 theo_occ = [p_empty; p_busy_bay_1; p_busy_bay_2; p_full];
7 figure;
8 bar([theo_occ, sim_occ]); %two bars per occupancy level
9 xlabel('Number of busy bays', 'FontSize', 14);
10 ylabel('Long-term probability', 'FontSize', 14);
11 title('J(0:3,5) - System occupancy levels: theory (v) vs. simulated (
    started empty)', 'FontSize', 16);
12 legend('Theory (v)', 'Simulated - started empty [0 0 0 0 0]', 'FontSize'
    , 12);

```

C.3.4 Busy bays and system occupation

```

1 mean_busy_bays = 0*p_empty + 1*p_busy_bay_1 + 2*p_busy_bay_2 + 3*p_full;
2 utilisation = mean_busy_bays / capacity;

```

C.4 Amplified arrival rate

```

1 valid = ~isnan(y_ampli);
2 figure;
3 plot(amplified(valid), y_ampli(valid), '-o', 'LineWidth', 1.5, '
    MarkerSize', 5);
4 hold on;
5 % Mark the original estimated arrival rates
6 xline(1,'--', 'Original estimated arrival probability', '
    LabelVerticalAlignment', 'bottom', 'FontSize', 12);
7 xlabel('Arrival-probability multiplier', 'FontSize', 14);
8 ylabel('Theoretical rejection percentage (\%)', 'FontSize', 14);
9 title('J(0:3,5) - Effect of amplified arrivals on rejection rate', '
    FontSize', 16);

```

C.5 Captured and missed revenue

```

1 valid_economic = ~isnan(captured_net_revenue) & ~isnan(
    missed_net_revenue);
2 figure;
3 plot(amplified(valid_economic), captured_net_revenue(valid_economic), -
    o , LineWidth , 1.5, MarkerSize , 5);
4 hold on;
5 plot(amplified(valid_economic), missed_net_revenue(valid_economic), -o
    , LineWidth , 1.5, MarkerSize , 5);
6 plot(amplified(valid_economic), missed_737_net_revenue(valid_economic),
    -o , LineWidth , 1.2, MarkerSize , 5);
7 plot(amplified(valid_economic), missed_e190_net_revenue(valid_economic),
    -o , LineWidth , 1.2, MarkerSize , 5);

```

```

8 hold off;
9 xline(1, '--', 'Original estimated arrival probability', '
    LabelVerticalAlignment', 'bottom', FontSize , 12);
10 xlabel('Arrival-probability multiplier', FontSize , 14);
11 ylabel('Estimated net revenue per model day (AUD)', FontSize , 14);
12 title('J(0:3,5) - Captured and missed net revenue', FontSize , 16);
13 legend('Captured (total)', 'Missed (total)', 'Missed - 737', 'Missed -
    e190', 'Location', 'northwest', FontSize , 12);

```

D Code quotes - Data

This appendix contains two sources of information, all of them processed using **PYTHON**.

Sources of information are divided according to its objective:

1. Database creation and cleaning: Are contained in 1. [BBDD conversor.ipynb](#)
2. Data analysis: Parameter estimations: Are contained in 2. [Arrival_analysis_v1.ipynb](#)

D.1 Database creation and cleaning:

D.1.1 Read data from .xlsl

Load package, Read data from excel (.xlsl), change the format to a list.

```

1 import pandas as pd #Load the package the process data
2 data_raw = pd.read_excel("ADL_Airport.xlsx", header=None)
3 data_raw.head()
4 data_list = data_raw[0].dropna().tolist()

```

D.1.2 Check data consistency

```

1 def is_flight_number(item):
2     flight_number = str(item).strip()
3     return len(flight_number) > 0 and flight_number[0].isalpha() and
4         flight_number[-1].isdigit() and '' not in flight_number
5 flights_raw = [] #Splitting into lists:
6 current = []
7 for item in data_list:
8     if is_flight_number(item) and current:
9         flights_raw.append(current)
10        current = [item]
11    else:
12        current.append(item)
13 if current:
14     flights_raw.append(current)
15 for f in flights_raw:
16     if len(f) != 5:
17         print(f'Showing unusual patterns are:\n - {len(f)} items: {f}')

```

D.1.3 Fixing identified source of length problems

```

1 flights_list = []
2 for f in flights_raw:
3     f = [x for x in f if not str(x).strip().lower().startswith('via')]
4     flights_list.append(
5         {'Flight': f[0].strip() if len(f) > 0 else '',
6         'Destination': f[1].strip() if len(f) > 1 else '',
7         'Estimated Time': f[2].replace("Est:", '').strip() if len(f) > 2
8         else ''},

```

```

8         'Gate': f[3].replace('Gate:', '').strip() if len(f) > 3 else ''
9         'Status': f[4].strip() if len(f) > 4 else '
          unknown'})
10 data = pd.DataFrame(flights_list)
11 data.head()

```

D.1.4 Inserting date

```

1 from datetime import date, timedelta
2 fixed_date = date(2026,7,26)
3 data['Date'] = fixed_date.strftime('%d/%m/%Y')
4 data.head()

```

D.1.5 Saving document

```

1 consolidated_db = 'ADL_aggregated_arrivals.csv'
2 import os
3 if os.path.exists(consolidated_db):
4     exists = pd.read_csv(consolidated_db)
5     if fixed_date.strftime('%d/%m/%Y') in exists['Date'].values:
6     else:
7         expected = len(flights_raw)
8         data.to_csv(consolidated_db, mode = 'a', header = False)
9 else:
10    data.to_csv(consolidated_db, mode = 'w', header = True)

```

D.2 Parameter estimation and analysis:

D.2.1 Categorisation

```

1 import pandas as pd
2 def categorise_aircraft(flight):
3     letters = ''.join(k for k in str(flight) if k.isalpha()).upper()
4     number = ''.join(k for k in str(flight) if k.isdigit())
5     if letters == "QF":
6         if len(number) == 3:
7             return "737"
8         if number.startswith("19") or number.startswith("17") or number.
9             startswith("18"):
10             return "e190"
11     if letters == "QQ":
12         if len(number) == 4:
13             return "e190"
14     if letters == "VA":
15         if len(number) == 3 or len(number) == 4:
16             return "737"
17     return "Insufficient information"

```

D.2.2 Access data and apply categorisation

```

1 data = pd.read_csv("ADL_aggregated_arrivals.csv")
2 data["Aircraft"] = data["Flight"].apply(categorise_aircraft)

```

D.2.3 Confirm planes and check amounts

```

1 landed = data["Status"].astype(str).str.strip().str.lower().eq("landed")
2 gated = data["Gate"].astype(str).str.strip().str.isdigit()

```

```

3 confirmed = data[landed & gated & data["Aircraft"].isin(["737", "e190"])
  ].copy()
4 confirmed["hour"] = pd.to_datetime(confirmed["Estimated Time"].astype(
  str).str.upper(),
5   format="%I:%M%p").dt.hour
6 days_count = confirmed["Date"].nunique()
7 b737_count = int((confirmed["Aircraft"] == "737").sum())
8 e190_count = int((confirmed["Aircraft"] == "e190").sum())
9 total_count = b737_count + e190_count
10 uncategorised_count = int((data.loc[landed&gated, "Aircraft"] == "
  Insufficient information").sum())
11 total_count2 = total_count + uncategorised_count
12 ratio_737 = (b737_count/total_count2)*100
13 ratio_e190 = (e190_count/total_count2)*100
14 concentration = ratio_737 + ratio_e190
15 status_clean = data["Status"].astype(str).str.strip().str.lower()
16 landed = status_clean.eq("landed")
17 cancelled = status_clean.str.contains("cancel")
18 other = ~(landed | cancelled)
19 print("Total database records:", len(data))
20 print("Landed records:", int(landed.sum()))
21 print("Cancelled records:", int(cancelled.sum()))
22 print("Other records:", int(other.sum()))

```

D.2.4 Arrival probability per beat, per plane

```

1 beat = 6 #Unit of measurement: minutes
2 hours_per_day = 24
3 mins_per_hour = 60
4 beats_per_day = int((hours_per_day * mins_per_hour) / beat)
5 p_beat_e190 = ((e190_count / days_count) / beats_per_day)
6 p_beat_737 = ((b737_count / days_count) / beats_per_day)
7 p_beat_no_arrival = 1 - (p_beat_e190 + p_beat_737)
8 last_date = data["Date"].iloc[-1]

```

E Economic Layer

n_ground_staff_level_2 = 8; n_ground_staff_level_3 = 5; n_engineer_level_3 = 1; cost_ground_staff_level_2 = 26.95; cost_ground_staff_level_3 = 27.54; cost_engineer_level_3 = 45.41;

transit_arrival_fee = 14.71; e190_seats_capacity = 90; b737_seats_capacity = 168; e190_occupancy_level = 1; b737_occupancy_level = 1;

E.1 Cost of services team

E.1.1 Cost per hour

```

1 estimated_cost_hour = (n_ground_staff_level_2 *
  cost_ground_staff_level_2) + (n_ground_staff_level_3 *
  cost_ground_staff_level_3) + (n_engineer_level_3 *
  cost_engineer_level_3);

```

E.1.2 Cost per beat

```

1 estimated_cost_beat = estimated_cost_hour * (beat/mins_per_hour);

```

E.1.3 Staff cost per aircraft

```
1 e190_estimated_team_cost_etc = estimated_cost_beat * e190_service_beats;  
2 b737_estimated_team_cost_etc = estimated_cost_beat * b737_service_beats;
```

E.2 Revenue - per arrival fee

```
1 e190_arrival_fee = e190_seats_capacity * e190_occupancy_level *  
    transit_arrival_fee;  
2 b737_arrival_fee = b737_seats_capacity*b737_occupancy_level*  
    transit_arrival_fee;
```

E.3 Net Revenue

```
1 e190_profit = e190_arrival_fee - e190_estimated_team_cost_etc;  
2 b737_profit = b737_arrival_fee - b737_estimated_team_cost_etc;
```

F Staff assumptions and wages

Used values are those given assuming **ordinary hours under a full-time contract**, without any penalty rates.

In the process of turnaround services, the following positions and levels of seniority associated are assumed:

- Ground services officer - Level 2: 8 people, this group has no additional responsibilities.

The functions are as follows:

- Cleaning team - "Fleet Presentation Agent": 3
- Ramp and baggage handling team - "Ramp Agent or Bag-room Agent": 3
- Watering and waste service: 1
- Catering: 1

- Ground services officer - Level 3: 5 The staff from this group has additional responsibilities as overseeing the development of tasks, distribute functions within their assigned teams, report, escalate or solve problems that may arise.

The functions are as follows:

- Cleaning team - "Fleet presentation team leader": 1.
- Ramp and baggage handling team - "Ramp team leader": 2.
- Catering - "Catering team leader": 2

- Professional engineer - Level 3: 1 person.

The staff from this group has additional responsibilities, as it is responsible for solving mechanical issues, small malfunctions of doors, water dispensers, toilets, seats and seat cover changes, and any other reported issue that may be solved within the turnaround service timings.

- Professional engineer - "Engineer on duty": 1.

Please note that the names of positions are only used as a reference in order to differentiate one specific function from the other.

A summary of the number of assumed hired staff and classification types is given in the Table below:

Classification type	Level	Staff number	Hourly rate (AUD)	Hourly subtotal (AUD)
Ground services officer	Level 2	8	26.95	215.60
Ground services officer	Level 3	5	27.54	137.70
Professional engineer	Level 3	1	45.41	45.41
Total	-	14	-	398.71

Table 21: Hired staff rates details.

G Transition Matrix Heatmap

In the following heatmaps, it can be seen which transitions are more likely to happen the brighter the colour gets, and which ones are unlikely, coloured in white.

Part A heatmap in Figure 12:

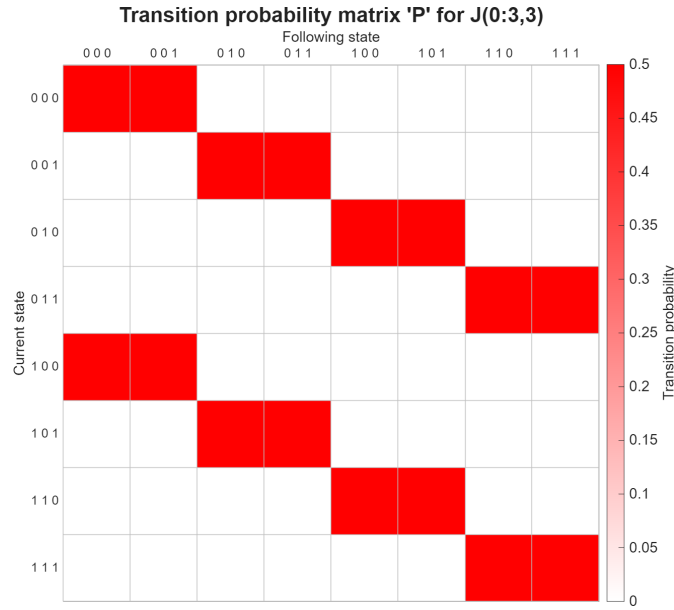


Figure 12: Part A - Transition matrix Heatmap

Figure 12 represents the 64 potential interactions between states in the 8×8 transition matrix, from which, only 16 are valid transitions, each with 0.5 probabilities of happening.

Part B heatmap in Figure 13:

Figure 13 represents the 1521 potential interactions between states in the 39×39 transition matrix, from which only 100 are valid transitions, each of them having different probabilities of happening varying from 1 to 0, the brighter the colour gets, the more likely is from that particular transition of happening.

H Graph diagram

I Eigenvalue Graphs

The following plot was generated to visually check the model's eigenvalues from the code in Appendix C.1.1 lines 3-9, as follows:

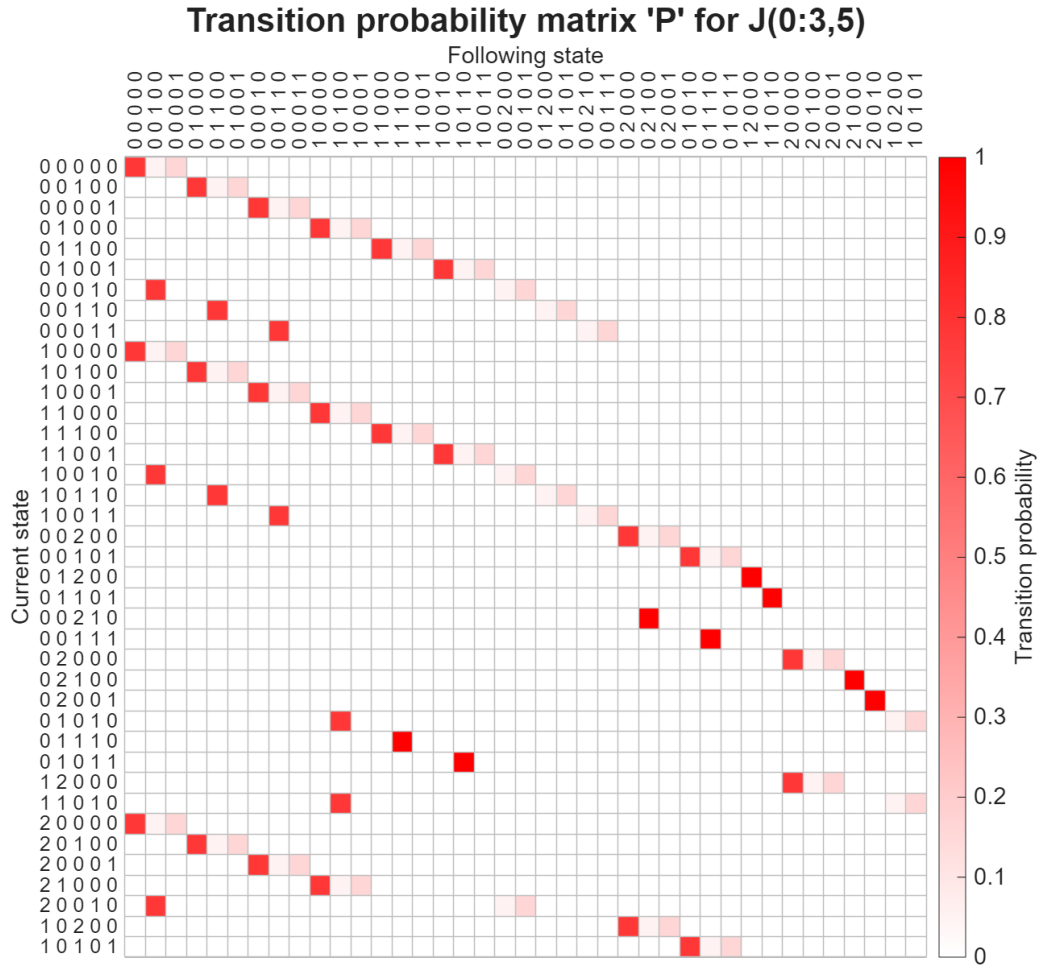


Figure 13: Part B - Transition matrix Heatmap

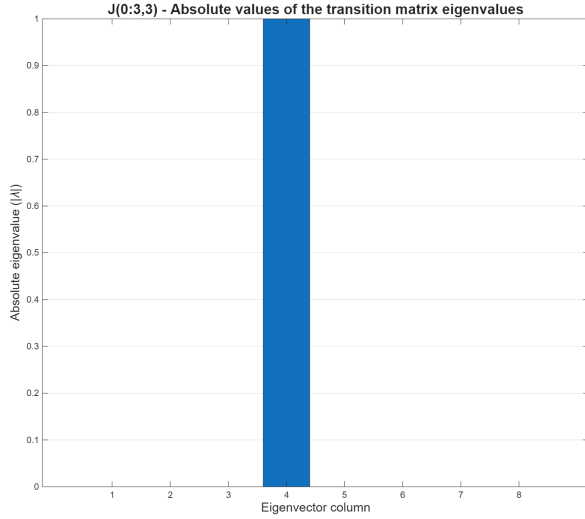


Figure 15: Part A - J(0:3,3) Values of models eigenvalues.

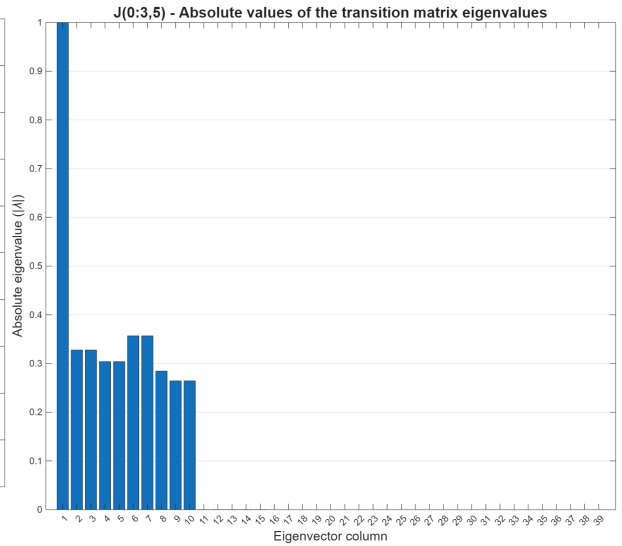


Figure 16: Part B - J(0:3,5) Values of models eigenvalues.

Figures 15 and 16 reveal that both models have only **one eigenvalue equal to 1**, eigenvectors columns 4 and 1, respectively. Consequently, both models have **unique** stationary distributions.

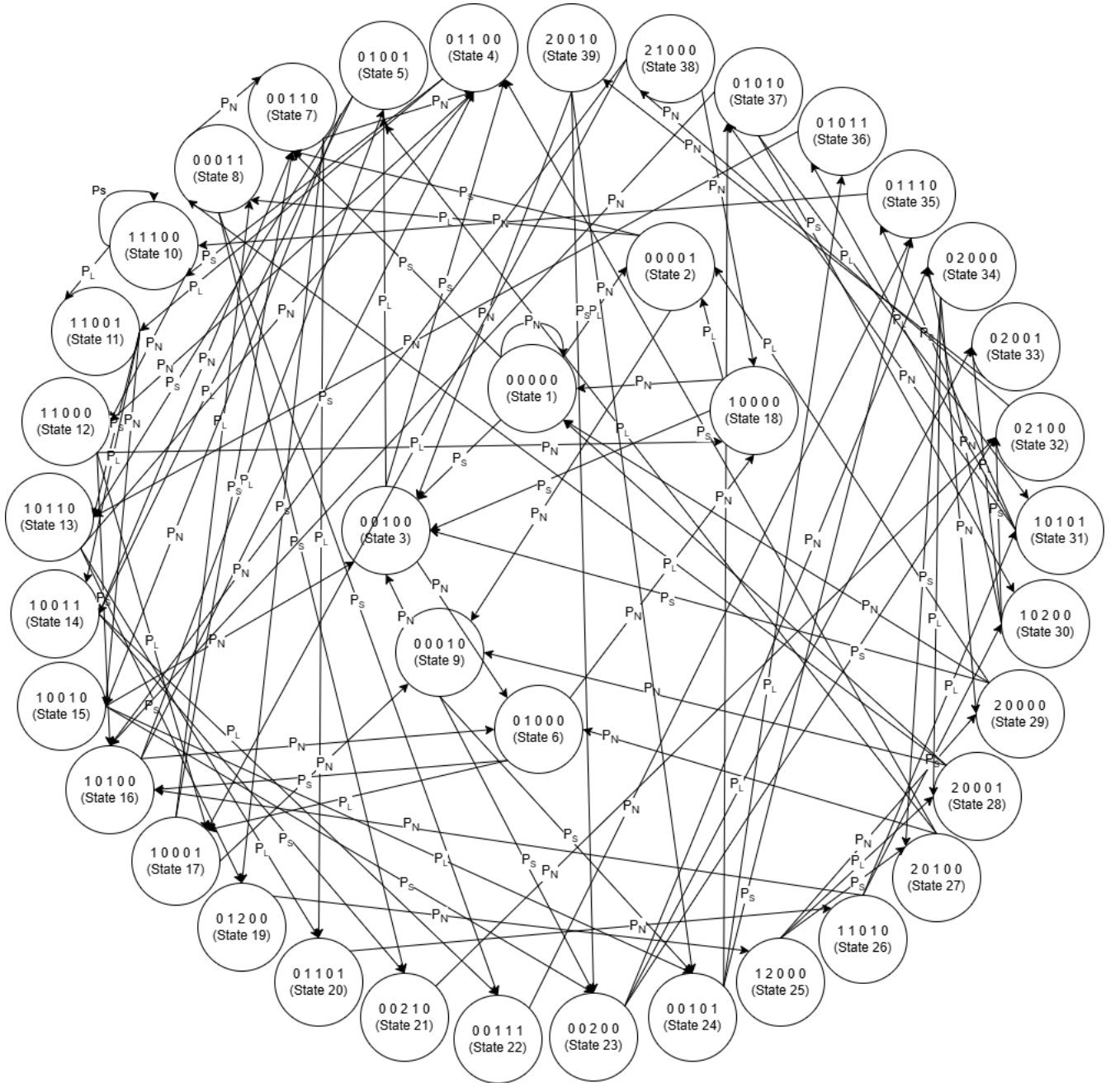


Figure 14: $J(0:3,5)$ - graph representation